**Федеральное государственное бюджетное образовательное учреждение высшего образования**
**«РОССИЙСКАЯ АКАДЕМИЯ НАРОДНОГО ХОЗЯЙСТВА и ГОСУДАРСТВЕННОЙ СЛУЖБЫ при Президенте Российской Федерации»**

**КОЛЛЕДЖ МНОГОУРОВНЕВОГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ**

**Специальность** 09.02.07 Информационные системы и программирование

УТВЕРЖДАЮ
Зам. директора КМПО РАНХиГС
_____С.Ф. Гасанов
«_____» _____ 2026 г.

# ДИПЛОМНЫЙ ПРОЕКТ

## на тему:

# «Программа для комплексного учёта расходов и доходов малого бизнеса. Модуль планирования и контроля бюджета»

Выполнил студент группы 408ИС-22 _____ Е.А. Самаричева

Руководитель _____ И.А. Фролкова

Нормоконтролер _____ И.А. Фролкова

Консультант по технико-экономическому обоснованию проекта _____ М.Р. Мирзоев

Старший консультант _____ О.А. Калашникова

**МОСКВА**
**2026 г.**

**СОДЕРЖАНИЕ**

ВВЕДЕНИЕ ... 3
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ ... 6
1.1 Описание предметной области и ее ключевых процессов ... 6
1.2 Формализация бизнес-процессов ... 8
1.3 Обзор существующих программных продуктов в рассматриваемой предметной области ... 11
1.4 Обоснование необходимости разработки нового программного продукта. Постановка задачи автоматизации ... 14
1.5 Выводы и выбор направлений разработки ... 17
2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ МОДУЛЯ ПЛАНИРОВАНИЯ И КОНТРОЛЯ БЮДЖЕТА ... 21
2.1 Предпроектное исследование ... 21
2.2 Разработка технического задания (ТЗ) на разработку программного продукта ... 24
2.3 Архитектурное проектирование ... 29
2.4 Реализация проекта ... 36
2.5 Тестирование ... 41
2.6 Выводы по проектированию и реализации проекта ... 43
3 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ... 46
3.1 Расчет затрат на разработку ... 46
3.2 Расчет показателей экономической эффективности ... 48
3.3 Выводы и обоснование целесообразности внедрения разработанного продукта ... 51
ЗАКЛЮЧЕНИЕ ... 53
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ... 55
ПРИЛОЖЕНИЕ А ... 60
ПРИЛОЖЕНИЕ Б ... 61
ПРИЛОЖЕНИЕ В ... 63
ПРИЛОЖЕНИЕ Г ... 68
ПРИЛОЖЕНИЕ Д ... 71

# ВВЕДЕНИЕ

В условиях современной цифровой экономики малые предприятия сталкиваются с необходимостью эффективно управлять своими финансовыми ресурсами, обеспечивать прозрачность бухгалтерского учета и повышать качество управленческих решений. Эффективный учет доходов и расходов является одним из ключевых факторов устойчивого развития бизнеса, поскольку позволяет отслеживать финансовые потоки, планировать будущие расходы и своевременно выявлять отклонения от запланированных показателей.

В настоящее время значительная часть малых предприятий использует разрозненные инструменты учета, такие как электронные таблицы, базовое бухгалтерское программное обеспечение или даже ручной учет. Такой подход приводит к ряду проблем, включая дублирование данных, ошибки при вводе, сложности с анализом финансовых показателей и отсутствие мониторинга исполнения бюджета в режиме реального времени. В результате снижается точность финансового планирования и возрастает риск принятия неэффективных управленческих решений.

Современное состояние рассматриваемой проблемы характеризуется наличием большого количества программных решений для учета финансов, однако большинство из них либо ориентировано на крупный бизнес, либо обладает избыточным функционалом, сложным интерфейсом и высокой стоимостью внедрения. В то же время специализированные решения, адаптированные под задачи малого бизнеса, зачастую не обеспечивают достаточного уровня аналитики и контроля, особенно в части сравнения плановых и фактических показателей. Это определяет необходимость разработки прикладных программных средств, ориентированных на конкретные задачи учета, планирования и контроля бюджета.

Актуальность темы дипломного проекта обусловлена потребностью малого бизнеса в доступном, удобном и функциональном инструменте, позволяющем не только вести учет доходов и расходов, но и осуществлять

планирование бюджета, а также контролировать отклонения фактических показателей от запланированных значений. Реализация такого решения позволяет повысить прозрачность финансовой деятельности, снизить вероятность ошибок и обеспечить более эффективное управление ресурсами предприятия.

Основанием для разработки темы дипломного проекта являются задачи автоматизации финансового учета, возникающие в деятельности малых предприятий, а также необходимость интеграции функций планирования и контроля в рамках единой информационной системы. В качестве исходных данных рассматриваются типовые процессы учета доходов и расходов, структура финансовых показателей предприятия, а также требования к хранению и обработке плановых и фактических значений.

Целью дипломного проекта является разработка программного модуля для комплексного учета доходов и расходов малого бизнеса, обеспечивающего планирование бюджета и контроль отклонений фактических данных от запланированных значений.

Для достижения поставленной цели необходимо решить следующие задачи:

– провести анализ предметной области и существующих решений в области финансового учета,

– сформировать требования к разрабатываемой системе,

– спроектировать архитектуру программного решения,

– разработать структуру базы данных для хранения плановых и фактических показателей,

– реализовать функционал формирования бюджета,

– обеспечить механизм сравнения плановых и фактических данных,

– реализовать расчет и отображение отклонений,

– провести тестирование разработанного программного модуля.

Объектом исследования являются система учета доходов и расходов малого бизнеса.

Предметом исследования выступают процесс финансового планирования и контроля бюджета.

Новизна проекта заключается в разработке программного решения, объединяющего функции учета, планирования и контроля в рамках единого модуля, ориентированного на потребности малого бизнеса. В отличие от существующих решений, особое внимание уделяется механизму сопоставления плановых и фактических показателей, а также удобству визуального представления отклонений, что повышает эффективность анализа финансового состояния предприятия.

Связь данного проекта с другими исследованиями проявляется в использовании существующих подходов к автоматизации учета и финансового анализа, а также в развитии концепций бюджетирования и контроля, применяемых в современных информационных системах. При этом в рамках дипломного проекта данные подходы адаптируются под условия малого бизнеса и реализуются в виде специализированного программного модуля.

Практическая значимость проекта заключается в возможности использования разработанной программы в деятельности малых предприятий для повышения точности учета, улучшения качества планирования и оперативного контроля финансовых показателей. Внедрение такого решения позволяет сократить трудозатраты на ведение учета, минимизировать ошибки и повысить обоснованность принимаемых управленческих решений.

Структура дипломного проекта определяется поставленной целью и включает введение, три главы, заключение и список использованных источников. В первой главе рассматриваются особенности предметной области и анализируются существующие решения, во второй главе выполняется проектирование программного модуля, а в третьей главе описывается разработка, реализация и тестирование системы.

# 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

## 1.1 Описание предметной области и ее ключевых процессов

В рамках дипломного проекта рассматривается деятельность малого предприятия по управлению денежными средствами. Эта деятельность напрямую влияет на жизнеспособность бизнеса, поскольку включает отслеживание доходов, контроль расходов и поиск точек для роста. Под объектом автоматизации здесь понимается совокупность всех финансовых потоков компании, а предметом – методы и программные инструменты, которые позволяют вести учёт более эффективно.

Ключевые бизнес-процессы в этой области можно разделить на три последовательных этапа. Первый – фиксация операций, то есть регистрация каждой хозяйственной операции с указанием даты, суммы и статьи дохода или расхода. Второй – группировка и аналитика, то есть классификация операций по категориям для последующего контроля. Третий, самый важный с точки зрения данной работы – планирование (бюджетирование). Бюджетирование представляет собой ключевой элемент управления, который обеспечивает координацию и контроль за деятельностью организации[10]. Завершающим этапом выступает контроль исполнения – сопоставление фактических данных с планом, расчёт отклонений и анализ причин их возникновения.

Несмотря на очевидную пользу такого подхода, многие малые предприятия сталкиваются со сложностями при постановке полноценного финансового учёта [32]. В литературе и на практике выделяют несколько системных недостатков, характерных для сложившегося порядка ведения дел. Прежде всего, это разрозненность данных: информация о выручке может храниться в одних файлах, чеки поставщиков – в других, а банковские выписки – в третьих. Собрать разрозненные сведения в единый отчёт становится трудоёмкой задачей. Кроме того, при ручном ведении учёта, например в Excel, высока вероятность ошибок: одна опечатка в формуле или неверно скопированная сумма способна исказить всю картину доходов и расходов [30]. Также часто наблюдается разрыв между планированием и

фактом – планы существуют отдельно от реальных цифр, и их сравнение приходится делать вручную, что не позволяет оперативно заметить перерасход или падение доходов. Еще одной проблемой является отсутствие наглядности: даже после сбора данных их сложно интерпретировать без графиков и предупреждающих индикаторов. Наконец, отсутствие стандартизации в отношении периодов планирования (месяц, квартал, год) приводит к тому, что каждый раз приходится создавать новую структуру файлов, что сбивает пользователей с толку и затрудняет перенос данных из одного периода в другой.

Анализ литературы и бухгалтерской практики в малых предприятиях позволяет определить информационные потребности пользователей. Им требуется простота ввода данных — возможность быстро устанавливать целевые показатели для каждой категории всего за несколько кликов. Также необходима единая панель управления, где фактические данные обновляются автоматически, а разница между целевыми и фактическими показателями видна сразу. Важна визуализация результатов: автоматический расчет отклонений, цветовая кодировка рисков (зеленый – норма, желтый – предупреждение, красный – критическое отклонение) и отображение тенденций в виде графиков.

Пользователи ожидают гибкости планирования – удобного переключения между месяцами, кварталами и годами без потери данных. Нужны и функции автоматизации рутины, например копирование лимитов из прошлого месяца или загрузка типового шаблона бюджета. И наконец, в условиях роста бизнеса возникает потребность в работе нескольких пользователей одновременно с разграничением прав доступа (бухгалтер, финансовый директор, администратор).

Ручные методы учёта и даже электронные таблицы уже не справляются с современными требованиями к скорости обработки информации и качеству контроля [30]. Существующие на рынке универсальные продукты либо слишком громоздки и дороги для малого бизнеса, либо не поддерживают

глубокую интеграцию планирования и факта. Поэтому в рамках дипломного проекта поставлена задача разработать специализированный программный модуль для автоматизации бюджетирования, который будет интегрирован в общую систему учёта доходов и расходов. Такой модуль должен обеспечить сквозной процесс – от постановки плановых задач до анализа их исполнения.

### 1.2 Формализация бизнес-процессов

Чтобы будущий модуль планирования и контроля бюджета работал именно так, как нужно пользователю, его бизнес-процессы необходимо описать формально. Формализация помогает перевести словесные описания в наглядные схемы, которые затем ложатся в основу программирования. В дипломном проекте для этой цели используются две нотации: BPMN для описания бизнес-процесса с ролями участников и UML-диаграмма деятельности для детализации алгоритма работы модуля. BPMN отвечает на вопросы: где начинается процесс, какие шаги выполняются, где принимаются решения, кто за что отвечает и где процесс завершается.

Для процесса «Управление бюджетом» построена BPMN-диаграмма, представленная в приложении А. На ней показаны два основных участника: пользователь (владелец бизнеса или финансовый менеджер) и система (программный модуль). Процесс начинается с того, что пользователь выбирает период планирования (месяц, квартал, год или произвольный интервал). Затем система загружает из базы данных имеющиеся бюджеты и фактические суммы операций. После этого пользователь может либо просмотреть отчёт «План vs Факт», либо выполнить действия по управлению бюджетом: добавить новый лимит, отредактировать существующий, удалить, скопировать из прошлого месяца, сохранить шаблон или загрузить его. Каждое из этих действий представляет собой отдельную задачу, которая выполняется пользователем с помощью системы. Диаграмма BPMN наглядно иллюстрирует последовательность и варианты ветвления процесса. Например, если пользователь добавляет бюджет, система проверяет наличие дубликатов и, в случае успеха, сохраняет запись, после чего возвращается к экрану выбора

периода. На диаграмме отображаются все ключевые события (начало, конец и промежуточные события, такие как «ошибка проверки»).

Такое моделирование дает представление о границах проектируемого модуля, о том, какие функции в нем реализованы, и в каком порядке они выполняются. В отличие от статической диаграммы вариантов использования, BPMN иллюстрирует динамику процесса, что особенно полезно для понимания логики будущего программного модуля.

Далее строится диаграмма действий для ключевого процесса, который можно назвать «Управление бюджетом: планирование и контроль» [11]. Она показывает последовательность шагов, выполняемых пользователем и системой, начиная с выбора периода и заканчивая отображением отчета. Вкратце алгоритм выглядит так. Пользователь выбирает тип периода (месяц, квартал, год или произвольный) и конкретные значения – например, месяц и год, квартал и год или две даты. Система отображает выбранный интервал. После нажатия кнопки «Обновить» система загружает из базы данных все бюджетные записи, попадающие в этот интервал, а также собирает фактические суммы из таблицы операций. Далее для каждой категории вычисляется плановая сумма, т.е. суммируются все бюджеты по данной категории в пределах периода, фактическая сумма и отклонение. По знаку отклонения определяется статус – для доходов положительное отклонение означает «Выполнено», отрицательное – «Отставание»; для расходов положительное отклонение – это «Перерасход», отрицательное – «В норме». Результаты выводятся в таблицу, на панель метрик и на столбчатую диаграмму «План vs Факт». Если пользователь решает добавить или отредактировать бюджет, открывается диалоговое окно, где он вводит сумму и, для длинных периодов, выбирает способ распределения: либо установить единую сумму на весь период, либо разбить её поровну на месяцы. Система проверяет, нет ли уже бюджета на эту категорию и период, и при успешной проверке сохраняет данные в базу, после чего обновляет отображение. Диаграмма деятельности, представленная на рисунке 1 наглядно показывает, что процесс является

циклическим: после каждого изменения бюджета происходит перерасчёт и обновление отчёта.

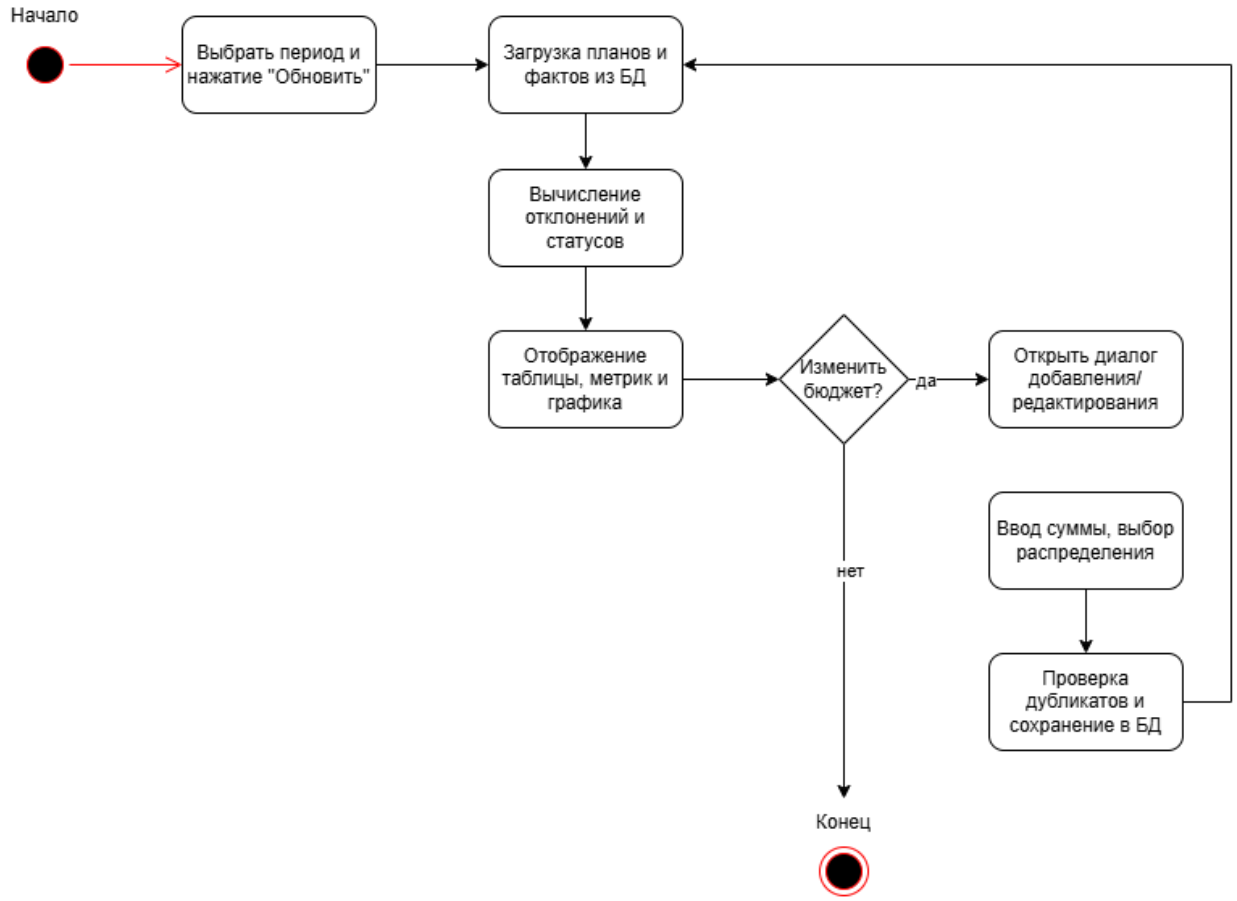



Рисунок 1 – UML диаграмма действий

Помимо графических диаграмм, полезно кратко описать два основных потока: «Планирование бюджета» и «Контроль исполнения». Поток планирования начинается с того, что пользователь выбирает категорию и период. Система проверяет, существует ли уже бюджет на эту пару. Если существует, выводится предупреждение, и пользователю предлагается перейти к редактированию. Если нет, новая запись сохраняется. При выборе года или квартала пользователь может либо задать одну сумму на весь длинный период, либо разбить сумму на месяцы – в последнем случае система автоматически создаёт несколько бюджетных записей, по одной на каждый месяц. После сохранения таблица бюджетов обновляется. Поток контроля исполнения не требует активных действий от пользователя, кроме выбора периода. Система самостоятельно собирает фактические суммы по категориям, сопоставляет их с плановыми и формирует отчёт. Фактические

суммы берутся из детализированных записей операций (таблица transaction_items). Это важно, потому что одна транзакция может быть распределена по нескольким категориям, и каждая из них будет учтена правильно. Итоговый отчёт включает таблицу с семью колонками: категория, тип, план, факт, отклонение, процент выполнения (в виде прогресс-бара) и статус.

Построенные диаграммы и описание потоков позволяют однозначно интерпретировать требования к модулю и служат основой для проектирования архитектуры и интерфейса. Формализация показала, что модуль должен поддерживать гибкий выбор периода, автоматический расчёт отклонений, цветовую индикацию состояний и работу с шаблонами.

### 1.3 Обзор существующих программных продуктов в рассматриваемой предметной области

На рынке представлено довольно много решений для учёта доходов и расходов. Однако если присмотреться, окажется, что большинство из них либо слишком сложны для небольшого предприятия, либо, наоборот, не умеют делать главное – помогать планировать бюджет и контролировать его исполнение. Рассмотрим несколько наиболее характерных примеров.

Первая группа – это мощные учётные системы, такие как «1С:ERP», «1С:Управление небольшой фирмой» и «1С:Бухгалтерия». Эти программы умеют практически всё: от проведения первичных документов до формирования сложной отчётности. В последних версиях появилась возможность получать фактические данные для бюджетирования прямо из оперативного учёта, контролировать лимиты доходов и расходов, а также настраивать разные сценарии планирования [34]. Для бизнеса, у которого уже выстроены сложные процессы, это серьёзный плюс. Но для малого предприятия такие возможности часто оказываются лишними – избыточный функционал только запутывает, а цена лицензии и стоимость внедрения могут достигать нескольких сотен тысяч рублей. Кроме того, чтобы просто

настроить план-фактный анализ, нужен квалифицированный специалист, и сам интерфейс таких программ человеку без подготовки освоить непросто.

Прямую противоположность представляют собой облачные сервисы, которые ориентированы на малый бизнес. Например, «Моё дело» или «Контур.Бухгалтерия». Они доступны через браузер, стоят недорого, и для начала работы достаточно базовых знаний компьютера. Автоматическое формирование отчётности, интеграция с банками, понятная поддержка – всё это сделано хорошо. Однако именно в части бюджетирования возможности таких сервисов часто ограничены. Они прекрасно помогают сдать отчётность в налоговую, но для того, чтобы, например, сравнить план и факт по конкретной статье расходов за прошлый квартал, пользователю, скорее всего, придётся выгружать данные в Excel и сводить их вручную. Как отмечает исследование ЮKassa, при выборе инструментов учёта предприниматели обращают внимание в первую очередь на формирование отчётов о движении средств и наличие календаря с планом прибыли и расходов [31]. Но именно эти функции в облачных бухгалтериях часто реализованы слабо – они существуют, но неудобны или не дают нужной детализации.

Существуют и специализированные сервисы для финансового планирования, например, «ПланФакт», «Финтабло» и «Finolog». В отличие от обычных бухгалтерий, они построены именно вокруг бюджетирования. В «Финтабло», к примеру, есть отдельный инструмент «Бюджет ДДС», который позволяет планировать поступления и списания по статьям, сравнивать план с фактом, видеть отклонения, копировать планы из прошлого месяца и даже делать прогнозы денежных потоков. Сервис ориентирован на малый и средний бизнес и работает по подписке, цены начинаются от нескольких тысяч рублей в месяц в зависимости от тарифа [35]. Но даже такие профильные системы имеют ограничения для тех, кто хочет полного контроля. Они всё равно навязывают определённую логику работы: как планировать, как вводить данные, как строить отчёты. К тому же, данные хранятся в облаке, и если

интернет пропадёт или сервис решит изменить тарифную политику, бизнес останется без доступа к своим финансовым планам.

Если говорить о бюджетном сегменте, есть ещё одна категория – бесплатные приложения. Для личных финансов это «ZenMoney», «HomeBank» и множество похожих. В HomeBank, например, можно задать бюджеты не только по крупным категориям, но и по подкатегориям – «Продукты» / «Рестораны», строить динамические отчёты и диаграммы [36]. Программа бесплатная, работает на разных платформах, и в ней действительно удобно планировать семейный бюджет. Но для бизнеса она не подходит: нет ни многопользовательского режима, ни интеграции с учётом хозяйственных операций, ни разграничения прав. Это хороший инструмент для личного пользования, но не для компании, где работает несколько человек и нужно вести полноценный учёт.

Таким образом, получается, что выбор для малого бизнеса невелик. Мощные учётные системы, такие как 1С, дороги и сложны. Облачные бухгалтерии не умеют глубоко работать с бюджетами. Специализированные сервисы по планированию лучше, но они работают по подписке, навязывают свои правила и зависят от интернета. А бесплатные приложения – это просто не для бизнеса.

Именно поэтому в рамках данного дипломного проекта было принято решение разработать собственный модуль планирования и контроля бюджета. Он будет встроен в уже существующую программу комплексного учёта доходов и расходов малого бизнеса. Пользователь сможет задавать планы по категориям, копировать их из прошлого месяца или шаблона, а система автоматически рассчитает фактические суммы и покажет отклонения. При этом все данные будут храниться локально в надёжной базе данных, никакой ежемесячной платы не потребуется. Такой подход сочетает в себе преимущества специализированных сервисов (глубокий план-фактный анализ) и простоту использования, необходимую для малого предприятия.

### 1.4 Обоснование необходимости разработки нового программного продукта. Постановка задачи автоматизации

Проведённый в предыдущих разделах анализ существующих решений для учёта и бюджетирования показывает, что ни одно из них в полной мере не подходит для малого предприятия, которому нужен простой, понятный и недорогой инструмент для планирования и контроля бюджета. У каждого из рассмотренных подходов есть свои недостатки, причём эти недостатки носят не случайный, а системный характер.

Если говорить об электронных таблицах, то их главная проблема — это человеческий фактор. Excel даёт предпринимателю гибкость и возможность быстро настроить учёт под себя, но вместе с тем он практически не защищает от ошибок. Как отмечает основатель компании-разработчика «РП-Интеграция» Фархад Баширов, у так называемой «Excel-автоматизации» есть как минимум семь ключевых недостатков [30]. В их числе — невозможность работать с файлами нескольких пользователей одновременно, отслеживать изменения, разграничивать права доступа и открывать отдельные окна с подробными сведениями при клике на строку таблицы. То есть даже если предприниматель добросовестно вносит все доходы и расходы в Excel, рано или поздно возникнет ситуация, когда кто-то из сотрудников случайно сотрёт формулу, введёт данные не в том формате или сохранит файл не в той папке. В таблицах Excel сложно обрабатывать большие объёмы данных, а контроль ввода и отслеживание изменений практически отсутствуют. Для учёта бюджета, где ошибка в одной ячейке может исказить картину выполнения плана за несколько месяцев, это особенно критично.

Мощные учётные системы, такие как 1С:УНФ, решают проблему контроля данных и многопользовательской работы [24]. В них действительно можно настроить бюджетирование и план-фактный анализ. Однако для малого бизнеса они оказываются слишком громоздкими и дорогими. Чтобы просто задать бюджет по категориям и увидеть отклонения, пользователю приходится разбираться в сложной иерархии справочников, настраивать сценарии

планирования, проекты и подразделения [24]. Как пишут сами разработчики 1С, создание бюджета в УНФ — это не отдельная лёгкая функция, а часть комплексной системы управленческого учёта [24]. Для небольшого предприятия, где штат может состоять из одного-двух человек, такой уровень сложности становится серьёзным препятствием. Более того, сама по себе автоматизация не решает проблемы управляемости, если учёт выстраивается без стратегического замысла, «под текущее положение дел».

Облачные сервисы, которые ориентированы на малый бизнес (такие как «Моё дело» или «Контур.Бухгалтерия»), тоже имеют свои ограничения в части бюджетирования [31]. Они хороши для бухгалтерского и налогового учёта, но инструменты для сравнения плана с фактом в них часто либо отсутствуют, либо реализованы слишком упрощённо. По данным исследования ЮKassa, 31% предпринимателей вообще не пользуются специализированным софтом, а многие из тех, кто пользуется, выбирают облачные сервисы прежде всего ради отчётности перед госорганами, а не ради управленческого анализа.

Специализированные сервисы для финансового планирования, такие как «Финтабло», действительно умеют планировать бюджет по статьям и сравнивать план с фактом [35]. У них продуманный интерфейс и интеграции с банками. Но и здесь есть минусы. Во-первых, это ежемесячная плата за подписку, которая для малого предприятия становится регулярной статьёй расходов. Во-вторых, отзывы пользователей показывают, что некоторые воспринимают «Финтабло» как «пустую трату денег на обычный эксель» — то есть даже в специализированном сервисе учёт может сводиться к ручному заполнению таблиц и дублированию информации [25]. В-третьих, все данные хранятся в облаке, а это означает зависимость от интернета и риски, связанные с политикой сервиса.

Таким образом, складывается парадоксальная ситуация. Excel — гибкий и бесплатный, но ненадёжный. 1С — надёжный и функциональный, но слишком сложный и дорогой для малого бизнеса. Облачные сервисы — простые и недорогие, но их возможности в части планирования бюджета

ограничены. Специализированные сервисы — хороши по функциям, но требуют регулярных платежей и привязывают бизнес к облаку.

Именно для преодоления этих недостатков и было принято решение разработать собственный программный модуль планирования и контроля бюджета. Он будет встроен в уже существующую программу комплексного учёта доходов и расходов малого бизнеса. Такой подход сочетает в себе лучшие черты рассмотренных решений и лишён их недостатков. Модуль не будет требовать ежемесячной оплаты, все данные будут храниться локально (никакой зависимости от интернета), а интерфейс останется простым и понятным для пользователя без специальной подготовки. При этом ключевая функция — автоматическое сравнение плановых и фактических показателей — будет реализована на должном уровне, с понятной визуализацией в виде таблиц, графиков и статусных индикаторов.

Пользователь должен иметь возможность выбирать период для анализа (месяц, квартал, год или произвольный интервал) и видеть сводную таблицу, где по каждой категории отражены плановая сумма, фактическая сумма, отклонение и статус выполнения. Добавление бюджета должно быть простым: пользователь выбирает категорию из справочника, вводит сумму, и система проверяет, нет ли уже бюджета на эту категорию и период. Для длинных периодов (год или квартал) должна быть предусмотрена функция автоматического распределения суммы по месяцам. Кроме того, пользователю нужны вспомогательные функции: копирование бюджетов из прошлого месяца, сохранение текущих лимитов в качестве шаблона и загрузка бюджета из шаблона. При просмотре отчёта «План vs Факт» система должна автоматически рассчитывать фактические суммы на основе зарегистрированных операций, вычислять отклонение по формуле «факт минус план» и определять статус с учётом типа категории (для доходов положительное отклонение — хорошо, для расходов — плохо). Все результаты должны визуализироваться в таблице с цветовой индикацией, на панели сводных метрик и на столбчатой диаграмме. Хранение данных должно

быть организовано в локальной базе данных, а интерфейс должен быть удобным и не требовать специального обучения.

Реализация всех перечисленных требований позволит малому предприятию отказаться от ручного сведения планов и фактов в Excel и LibreOffice, избежать ошибок при расчётах и оперативно получать наглядную картину исполнения бюджета, что в конечном счёте повысит качество управленческих решений.

### 1.5 Выводы и выбор направлений разработки

Проведённое в первой главе исследование позволяет сформулировать несколько взаимосвязанных выводов, которые не только обобщают полученные знания, но и задают вектор для всех последующих этапов проектирования и разработки.

Анализ показал, что учёт доходов и расходов в малом бизнесе не сводится к простой регистрации операций. Ключевым требованием является наличие сквозного процесса бюджетирования: от установления плановых лимитов по категориям до автоматического сопоставления с фактическими данными и расчёта отклонений. Без такого механизма предприятие теряет способность оперативно реагировать на перерасход средств или падение выручки, что критично для ограниченных финансовых ресурсов. Изучение типовых процессов (фиксация операций, группировка, планирование, контроль) выявило системные недостатки ручного и полуавтоматического учёта: дублирование данных, ошибки ввода, отсутствие единой базы для сравнения плана и факта, сложности с переносом данных между периодами. Эти проблемы напрямую снижают качество управленческих решений и создают риски для бизнеса.

Обзор рынка был проведён по четырём основным классам продуктов, и каждый из них продемонстрировал принципиальные ограничения с точки зрения малого бизнеса. Электронные таблицы (LibreOffice и аналоги) обладают гибкостью и нулевой стоимостью входа, но не обеспечивают защиты от ошибок, не поддерживают многопользовательский режим, не

имеют разграничения прав доступа и аудита изменений. Как показано в исследовании Фархада Баширова, «Excel-автоматизация» ведёт к потере данных, сложности обработки больших объёмов и отсутствию контроля версий. Для бюджетирования, где ошибка в одной формуле может исказить результаты за несколько месяцев, такой подход неприемлем. Мощные учётные системы (1С:ERP, 1С:УНФ, 1С:Бухгалтерия) безусловно, решают задачи контроля и многопользовательской работы. Однако их функционал бюджетирования перегружен настройками (сценарии, проекты, подразделения), требует квалифицированного внедрения и сопровождается высокими лицензионными затратами (сотни тысяч рублей). Для малого предприятия со штатом 1–3 человека такая сложность становится непреодолимым барьером, а регулярное обновление и сопровождение – дополнительным бременем. Облачные сервисы для сдачи отчётности («Моё дело», «Контур.Бухгалтерия») удобны и недороги для налогового учёта, но инструменты план-фактного анализа в них либо отсутствуют, либо реализованы на примитивном уровне. Пользователю приходится выгружать данные в Excel и сводить их вручную, что возвращает к проблемам первого класса. Исследование ЮKassa подтверждает, что 31% предпринимателей не используют специализированный софт именно из-за нехватки нужных функций. Специализированные сервисы для бюджетирования («Финтабло», «ПланФакт») наиболее близки к потребностям, предлагая глубокий план-фактный анализ, шаблоны, копирование периодов. Однако они работают по ежемесячной подписке (от нескольких тысяч рублей), хранят данные в облаке (зависимость от интернета и риски доступа) и, по отзывам пользователей, иногда воспринимаются как «очередной Excel за деньги». Кроме того, их логика работы жёстко предопределена, что ограничивает адаптацию под конкретные предприятия.

Ни один из существующих инструментов не предлагает локального, бесплатного (без абонентской платы), простого в освоении и при этом функционально полноценного решения для бюджетирования,

интегрированного с оперативным учётом доходов и расходов. Рынок имеет явный пробел, который и призван заполнить данный дипломный проект.

Построенные BPMN-диаграмма процесса управления бюджетом и UML-диаграмма деятельности позволили не просто описать, но и строго алгоритмизировать действия пользователя и системы. Формализация выявила следующие типовые сценарии, которые обязана поддерживать система:

– выбор периода (месяц, квартал, год, произвольный интервал);

– просмотр сводной таблицы «План vs Факт» с автоматическим расчётом отклонений и статусов (с учётом типа категории – доход или расход);

– добавление, редактирование, удаление бюджетных записей;

– проверка на дублирование (категория + период);

– для длинных периодов – автоматическое распределение суммы по месяцам (единым лимитом или равными долями);

– копирование бюджетов из прошлого месяца;

– сохранение текущих лимитов как шаблона и загрузка из шаблона.

Диаграммы также показали, что процесс является циклическим: после каждого изменения бюджета система должна пересчитывать фактические суммы и обновлять отчёт. Это накладывает требование к производительности и корректной агрегации данных из таблиц операций (transactions и transaction_items). Формализация подтвердила, что разработка отдельного модуля – технически выполнимая задача, а описанные потоки могут быть реализованы в реляционной базе данных с использованием SQL-запросов группировки и фильтрации.

Синтезируя результаты анализа, можно утверждать, что существующие решения не удовлетворяют трём ключевым критериям малого бизнеса: доступность (низкая стоимость владения), автономность (работа без постоянного интернета и без ежемесячных платежей) и простота использования (интуитивный интерфейс без обучения). Разрабатываемый модуль, встроенный в программу «Финанс Про», призван объединить лучшие качества специализированных сервисов (глубокий план-фактный анализ,

шаблоны, копирование) с надёжностью локальной базы данных и отсутствием регулярной абонентской платы. Кроме того, предложенное решение сохраняет полный контроль над данными внутри компании, что критично с точки зрения коммерческой тайны.

Проведённый анализ предметной области доказал актуальность темы, выявил неудовлетворённые потребности малого бизнеса в инструменте бюджетирования, формализовал требования и выбрал оптимальный технологический стек. Разработка модуля планирования и контроля бюджета является не только научно-обоснованной, но и практически востребованной.

# 2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ МОДУЛЯ ПЛАНИРОВАНИЯ И КОНТРОЛЯ БЮДЖЕТА

## 2.1 Предпроектное исследование

Главная задача модуля планирования – дать пользователю возможность управлять бюджетом: задавать плановые суммы по категориям доходов и расходов, а затем видеть, насколько фактические показатели соответствуют этим планам. Исходя из анализа предметной области и потребностей пользователей, можно сформулировать следующий набор функциональных требований [19], [21].

Модуль должен позволять выбирать период анализа (месяц, квартал, год или произвольный интервал) и отображать таблицу, в которой для каждой категории указаны плановая сумма, фактическая сумма и отклонение (разница между фактом и планом). Пользователь должен иметь возможность добавлять новые бюджетные записи – выбирать категорию из справочника, указывать период и вводить сумму. При этом система обязана проверять, нет ли уже бюджета на ту же категорию и тот же период, чтобы избежать дублирования. Если пользователь выбирает длинный период (год или квартал), модуль должен предложить два способа: либо задать одну сумму на весь период, либо автоматически разбить её поровну на месяцы. Во втором случае система сама создаст отдельные бюджетные записи для каждого месяца.

Также пользователю нужны вспомогательные функции, которые упрощают рутинную работу. Например, копирование бюджетов из прошлого месяца в текущий – это избавляет от необходимости заново вводить одинаковые лимиты каждый месяц. Сохранение текущих лимитов в качестве шаблона и последующая загрузка из шаблона позволяют быстро настроить бюджет на новый период, взяв за основу ранее утверждённые значения. Эти функции особенно полезны для малого бизнеса, где одни и те же статьи расходов (аренда, связь, закупка материалов) повторяются из месяца в месяц.

Помимо управления бюджетом, модуль должен предоставлять отчёт «План vs Факт». Этот отчёт формируется автоматически на основе

фактических сумм, которые берутся из уже существующей в программе базы операций. Система суммирует все доходы и расходы по категориям за выбранный период, затем вычисляет отклонение по формуле «факт минус план». Для доходов положительное отклонение – это хорошо (перевыполнение плана), поэтому статус должен быть «Выполнено», а для расходов положительное отклонение – плохо (перерасход), статус «Превышение». Соответственно отрицательное отклонение для расходов означает экономию – статус «В норме», а для доходов – «Отставание». Результаты должны быть наглядными: таблица с цветовой индикацией (например, красный цвет для перерасхода, зелёный для нормы), панель сводных метрик (общий план, общий факт, общее отклонение) и столбчатая диаграмма, где план и факт показаны рядом для каждой категории.

Главное для малого бизнеса – это надёжность и целостность данных. Для этого сформулирован набор нефункциональных требований. Модуль должен хранить все бюджетные записи в реляционной базе данных с поддержкой ссылочной целостности. Это означает, что нельзя просто так удалить категорию, если на неё ссылаются бюджеты или операции. Система должна предотвращать потерю данных и обеспечивать их согласованность. Поскольку программа работает локально, без постоянного подключения к интернету, база данных должна быть встроенной или легковесной, но при этом поддерживать многопользовательский доступ – на случай, если в компании несколько сотрудников ведут учёт одновременно.

Производительность тоже важна. Модуль должен обрабатывать запросы к базе данных за разумное время. При загрузке бюджета за год, когда может быть несколько сотен записей, отклик системы не должен превышать одной-двух секунд. Расчёт фактических сумм требует агрегации по таблице операций, которая со временем может вырасти до десятков тысяч записей. Поэтому необходимо использовать индексы на ключевых полях (дата, категория, период) и оптимизировать SQL-запросы.

Безопасность в локальном приложении может показаться не самой острой темой, но пренебрегать ею не стоит. Данные о доходах и расходах – это коммерческая тайна. Модуль должен обеспечивать разграничение доступа: разные пользователи должны входить под своими учётными записями, и их права могут различаться. Например, бухгалтер может только просматривать отчёты, а финансовый директор – редактировать бюджеты. Кроме того, система должна вести журнал действий – кто, когда и что изменил в бюджетных записях. Это поможет отследить случайные или намеренные правки.

Удобство использования (usability) – отдельное и очень важное нефункциональное требование. Поскольку у малого бизнеса обычно нет штатного IT-специалиста, интерфейс должен быть интуитивно понятным. Пользователь не должен гадать, где найти ту или иную функцию. Переключение между периодами, добавление бюджета, просмотр отчёта – всё должно быть на виду. Важно минимизировать количество кликов для типовых операций. Например, чтобы скопировать бюджет из прошлого месяца, достаточно одной кнопки. Также интерфейс должен быть адаптивным – окно можно разворачивать на весь экран, таблицы должны прокручиваться, а шрифты – оставаться читаемыми.

И наконец, модуль должен быть расширяемым. В будущем могут появиться новые требования – например, интеграция с банковскими выписками или прогнозирование денежных потоков. Архитектура модуля должна позволять добавлять новые функции без полной переделки уже существующего кода. Это достигается за счёт модульной организации, разделения логики, данных и представления.

Предпроектное исследование позволило собрать полный набор требований, которые лягут в основу технического задания. Модуль должен быть функционально полным (позволять управлять бюджетом и контролировать исполнение), надёжным (целостность данных, работа без

потерь), производительным (быстрые запросы), безопасным (разграничение прав и аудит), удобным (интуитивный интерфейс) и расширяемым.

## 2.2 Разработка технического задания (ТЗ) на разработку программного продукта

По результатам предпроектного исследования составлено техническое задание на модуль «Планирование и контроль бюджета». Этот модуль входит в состав приложения «Финанс Про» и обеспечивает автоматизацию бюджетирования для малого бизнеса. Техническое задание определяет, для чего нужен модуль, какие задачи он решает, из каких этапов состоит разработка, а также как проверяется готовность продукта.

Модуль называется «Планирование и контроль бюджета». Он предназначен для того, чтобы пользователь (финансовый менеджер или владелец бизнеса) мог задавать плановые суммы по категориям доходов и расходов, отслеживать их исполнение, быстро копировать бюджеты из прошлых периодов и использовать шаблоны. Все результаты работы модуля отображаются в виде таблиц, сводных метрик и столбчатых диаграмм. Модуль работает в связке с другими частями «Финанс Про» (учёт операций, аналитика, справочники) через общую базу данных.

Разработать программный модуль, который позволяет легко устанавливать плановые суммы на любой период (месяц, квартал, год или произвольный интервал), автоматически получать отчёт об исполнении бюджета с расчётом отклонений (план-факт) и статусов, а также копировать бюджеты и применять шаблоны без ручного переноса данных в электронных таблицах.

Модуль должен давать возможность выбирать период анализа с помощью кнопок быстрых периодов или календаря. При добавлении бюджета пользователь выбирает категорию, указывает сумму и период. Система проверяет, нет ли уже бюджета на эту категорию и период. Если период длинный (год или квартал), можно либо задать одну сумму на весь период,

либо автоматически разбить её поровну по месяцам. Пользователь также может редактировать и удалять уже созданные бюджеты.

Для удобства работы реализовано копирование бюджетов из предыдущего месяца (когда выбран тип «Месяц») – все лимиты переносятся в текущий месяц без дублирования. Текущие лимиты можно сохранить как шаблон (используется специальный период 1999 год), а затем загрузить этот шаблон в любой другой период.

На вкладке «Исполнение» модуль автоматически рассчитывает фактические суммы на основе уже введённых операций (данные берутся из модуля учёта). Результат представляется в виде таблицы с колонками: категория, тип (доход/расход), план, факт, отклонение, процент выполнения, статус. Отклонение вычисляется по формуле «факт минус план». Для доходов положительное отклонение означает «Выполнено», отрицательное – «Отставание». Для расходов положительное отклонение – это «Перерасход», отрицательное – «В норме». Таблица снабжена цветовой индикацией, а также выводится панель сводных метрик (общий план, общий факт, общее отклонение) и столбчатая диаграмма сравнения плана и факта.

Для реализации модуля выбран язык Python [18]. Причин для этого несколько. Во-первых, Python имеет простой и читаемый синтаксис, что ускоряет разработку и облегчает сопровождение кода. Во-вторых, для Python существует огромное количество библиотек практически под любую задачу – от построения графиков до работы с базами данных. В-третьих, Python является кроссплатформенным: программа, написанная на нём, будет работать и на Windows, и на Linux, и на macOS. Для малого бизнеса это важное преимущество, так как неизвестно, какая операционная система используется на предприятии. Согласно исследованиям, Python остаётся одним из самых популярных языков для разработки прикладного программного обеспечения, особенно в сфере анализа данных и бизнес-приложений.

Теперь о фреймворке для создания графического интерфейса. Здесь выбор пал на PyQt6 [38]. Эта библиотека представляет собой набор привязок

к популярному фреймворку Qt. PyQt6 позволяет создавать современные, отзывчивые и визуально приятные интерфейсы. В отличие от многих других решений, PyQt6 даёт полный контроль над внешним видом окна, поддерживает стилизацию с помощью CSS-подобных таблиц стилей, а также включает множество готовых виджетов: таблицы, выпадающие списки, дата-редакторы, вкладки и так далее. Именно эти элементы нужны для модуля бюджетирования – таблица с планами и фактами, форма добавления бюджета, фильтры по периоду, вкладки «Планирование» и «Исполнение». Кроме того, PyQt6 хорошо сочетается с другими библиотеками, например, с Matplotlib для встраивания графиков прямо в интерфейс.

Для построения диаграмм и графиков, которые показывают сравнение плана и факта, используется библиотека Matplotlib [27]. Это стандартный инструмент для визуализации данных в экосистеме Python. Matplotlib позволяет создавать столбчатые диаграммы, круговые диаграммы, линейные графики и многое другое. В модуле она нужна для отображения столбчатой диаграммы «План vs Факт» по категориям, а также для дополнительных графиков, если потребуется. Matplotlib легко встраивается в PyQt через специальный виджет FigureCanvas.

Для работы с базами данных выбрана система управления базами данных MySQL [28]. Это одна из самых популярных реляционных СУБД с открытым исходным кодом. SQLite удобен для одно-пользовательских приложений, но он имеет ограничения при одновременной работе нескольких пользователей и при высоких нагрузках на запись. В малом бизнесе, возможно, учётом занимается один человек, но проект закладывает возможность расширения – если предприятие вырастет и в нём появится несколько финансовых сотрудников. MySQL поддерживает многопользовательский режим, транзакции, сложные запросы и хранимые процедуры, обеспечивает высокую надёжность и целостность данных. Кроме того, для взаимодействия Python с MySQL существует драйвер mysql-connector-python или библиотека SQLAlchemy, которая предоставляет

удобный объектно-реляционный интерфейс. SQLAlchemy позволяет не писать «сырые» SQL-запросы в коде, а работать с таблицами как с классами Python, что ускоряет разработку и снижает риск ошибок.

Среда разработки – это инструмент, в котором пишется код. Для данного проекта выбрана интегрированная среда разработки PyCharm от компании JetBrains. PyCharm имеет бесплатную версию Community, которая полностью подходит для образовательных и некоммерческих проектов. Эта среда поддерживает автодополнение кода, отладку, интеграцию с системами контроля версий (например, Git), а также удобные инструменты для работы с базами данных. В качестве альтернативы можно было бы использовать Visual Studio Code, но PyCharm считается более специализированным для Python-разработки.

Что касается тестирования, то в ходе разработки применяются как ручные, так и автоматизированные методы. Для модульного тестирования отдельных функций (например, расчёта отклонений, валидации ввода) используется фреймворк pytest [29]. Он позволяет быстро написать набор тестов и запускать их после каждого изменения кода, чтобы убедиться, что ничего не сломалось. Интеграционное тестирование (проверка взаимодействия модуля с базой данных) выполняется вручную с помощью предварительно подготовленных тестовых данных. Кроме того, в процессе разработки ведётся журнал ошибок, где фиксируются обнаруженные дефекты и способы их устранения. Такой подход соответствует современной практике разработки программного обеспечения, где тестирование является неотъемлемой частью жизненного цикла.

Модуль должен быть надёжным: все SQL-запросы параметризованы, что исключает SQL-инъекции, а при сбое подключения к базе данных программа не аварийно завершается, а выводит понятное сообщение. Производительность: загрузка годового отчёта (до 1000 бюджетных записей и 5000 транзакций) занимает не более двух секунд. Интерфейс интуитивно понятен, все кнопки снабжены всплывающими подсказками, для типовых

операций требуется не более трёх кликов. Архитектура позволяет добавлять новые отчёты и функции без переписывания существующего кода (разделение на слои представления, бизнес-логики и доступа к данным). Модуль корректно работает на Windows 10/11 и Linux.

В процессе создания модуля были выполнены следующие работы: настройка среды разработки и репозитория Git, реализация базы данных в соответствии с общей структурой (таблицы categories, budgets, transactions, transaction_items), создание классов Module2Widget (главный виджет), PlanFactTab (вкладка контроля исполнения), BudgetAddDialog и BudgetEditDialog (диалоги), написание SQL-запросов для загрузки бюджетов и агрегации фактических сумм, разработка механизмов копирования, шаблонов и проверки дубликатов, стилизация интерфейса и интеграция с системой авторизации, модульное тестирование (pytest) и интеграционное тестирование с тестовой базой данных, подготовка документации (листинги кода, скриншоты) [29].

Техническое задание, составленное на основе описанных требований и выбранных технологий, включает следующие ключевые пункты. Модуль должен быть реализован на Python 3.10 или выше, с использованием PyQt6 для интерфейса, MySQL в качестве СУБД и Matplotlib для графиков. Исходный код должен храниться в системе контроля версий Git. Тестирование проводится с помощью pytest. Разработка ведётся в среде PyCharm. Готовая программа должна представлять собой исполняемый файл (или набор файлов), который можно запустить на компьютере пользователя без установки дополнительных компонентов (с учётом наличия установленного Python и необходимых библиотек либо с использованием упаковщика вроде PyInstaller). Все данные пользователя (бюджеты, категории, операции) хранятся в базе данных MySQL, расположенной локально или на сервере в локальной сети.

Исходя из вышеперечисленного, техническое задание и выбор технологий полностью соответствуют требованиям, выявленным на этапе

предпроектного исследования. Python и PyQt6 обеспечивают простоту разработки и кроссплатформенность, MySQL даёт надёжное хранение данных с возможностью многопользовательского доступа, а Matplotlib позволяет наглядно визуализировать результаты план-фактного анализа. В следующем разделе будет рассмотрена архитектура программного решения – как все эти компоненты соединяются в единую работающую систему. Общее техническое задание на разработку программного продукта для учета доходов и расходов малого бизнеса «Финанс Про» представлено в приложении Д.

### 2.3 Архитектурное проектирование

В этом разделе описывается, как устроено приложение Финанс Про. Приложение разрабатывается группой из четырёх человек, каждый отвечает за свой модуль, поэтому архитектура должна быть чётко разделена, но при этом модули должны работать вместе через общую базу данных.

Финанс Про построен по клиент-серверной архитектуре с монолитным ядром [9]. Единое приложение-клиент (на Python с PyQt6) устанавливается на компьютеры пользователей и подключается к серверу баз данных MySQL. Это позволяет нескольким сотрудникам работать одновременно с одними и теми же данными. Внутри самого клиента каждый модуль логически выделен, но они собираются в одно исполняемое приложение. Такой подход называется монолитным, потому что все модули работают в одном процессе и обмениваются данными через вызовы методов, а не через сетевые запросы. Для малого бизнеса это проще и дешевле, чем микросервисы.

Схема компонентов модуля планирования и контроля бюджета на рисунке 2 отражает его внутреннюю структуру и взаимодействие с базой данных. Пользователь работает с графическим интерфейсом, который реализован в классе Module2Widget. Этот класс содержит две вкладки: «Планирование» (таблица бюджетов, кнопки добавления, редактирования, удаления, копирования, работы с шаблонами) и «Исполнение (План/Факт)» (класс PlanFactTab, который отображает сводные метрики, таблицу с отклонениями и столбчатую диаграмму). Для добавления и редактирования

бюджета используются диалоговые окна BudgetAddDialog и BudgetEditDialog, которые не обращаются к базе напрямую, а возвращают введённые пользователем данные.

Все запросы к базе данных выполняются через объект controller, который передаётся в Module2Widget при создании. Этот контроллер предоставляет методы execute для выполнения SQL-запросов, а также инкапсулирует логику доступа к таблицам budgets, categories, transactions и tr ansaction_items. Таким образом, интерфейс не знает деталей реализации запросов, а только вызывает методы контроллера и получает результаты. Контроллер, в свою очередь, взаимодействует с репозиторием (слоем доступа к данным), который формирует и отправляет запросы в базу данных MySQL. Такое разделение на уровни (интерфейс, контроллер, репозиторий, база данных) соответствует принципам модульной архитектуры и упрощает тестирование отдельных компонентов.

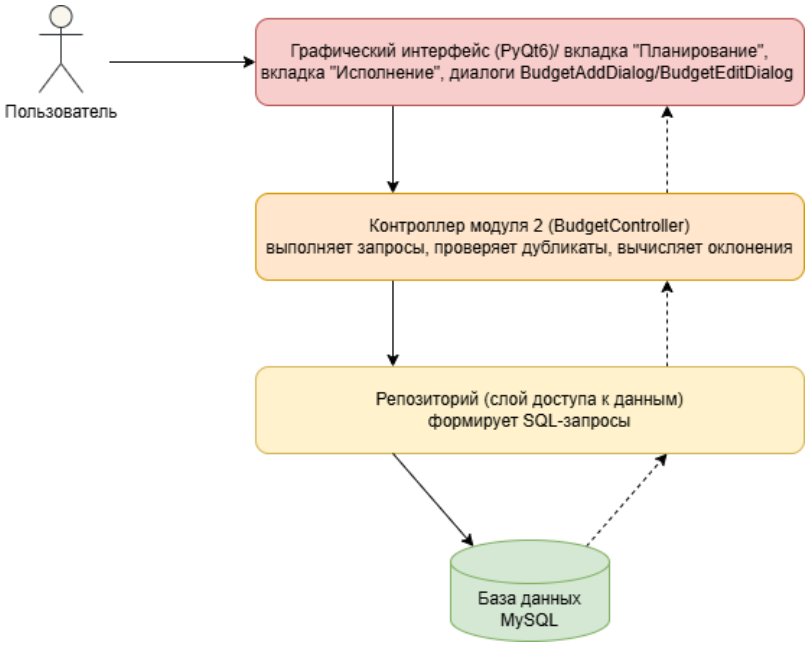



Рисунок 2 – Схема компонентов

ER-диаграмма рисунок 3 показывает основные сущности, которые хранятся в базе данных [22]. Это «Категория», «Бюджет», «Транзакция», «Элемент транзакции», «Пользователь», «Роль». Связи между ними следующие. У одной категории может быть много бюджетов (один-ко-многим). У одной категории может быть много элементов транзакций. У

одной транзакции может быть много элементов транзакций (это нужно для операций, которые относятся сразу к нескольким категориям – например, чек, где часть суммы потрачена на товары, а часть на доставку). Один пользователь может создавать много транзакций. Роль может быть назначена многим пользователям. Каждая сущность имеет свой набор атрибутов. Например, «Бюджет» включает период (дата начала и окончания) и плановую сумму. «Элемент транзакции» содержит сумму, привязанную к конкретной категории. Эта модель позволяет модулю планирования агрегировать фактические суммы по категориям и периодам.

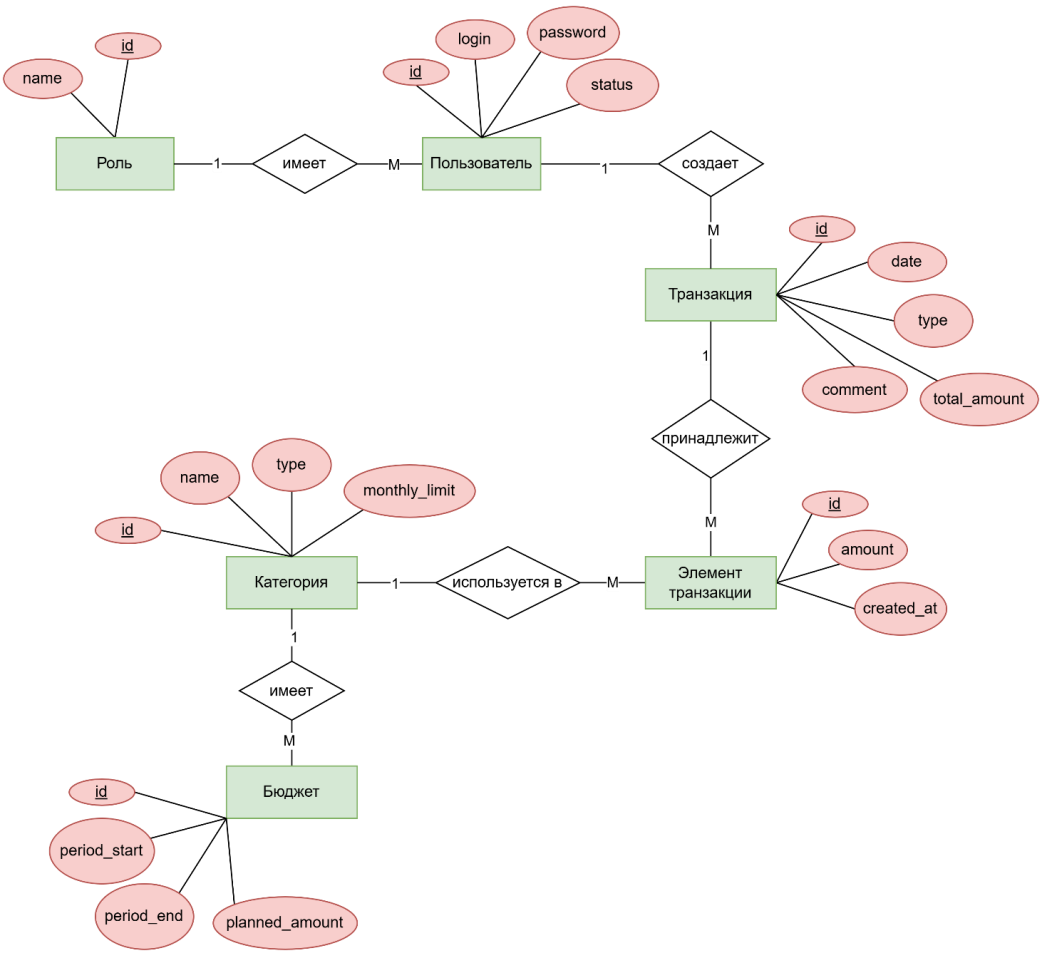



Рисунок 3 – ER-диаграмма

Логическая схема рисунке 4 детализирует ER-модель [22]. Таблица categories содержит поля id (первичный ключ), name, type (доход или расход), parent_id (внешний ключ на ту же таблицу для иерархии), monthly_limit. Таблица budgets – id, category_id (внешний ключ к categories.id), period_start, period_end, planned_amount. Таблица transactions – id, date, type, total_amount,

comment, user_id (внешний ключ к users.id), counterparty_id. Таблица transaction_items – id, transaction_id (внешний ключ к transactions.id), category_id (внешний ключ к categories.id), amount. Таблицы users и roles описывают пользователей и их роли. Физическая схема добавляет конкретные типы данных (например, DECIMAL(12,2) для денежных сумм, DATETIME для даты транзакции) и индексы. В MySQL для всех таблиц используется движок InnoDB, который поддерживает транзакции и ссылочную целостность. Индексы создаются на полях, которые часто участвуют в условиях WHERE и JOIN: на period_start и period_end в таблице budgets, на category_id в transaction_items, на date в transactions. Это ускоряет запросы, которые выполняет модуль бюджетирования.

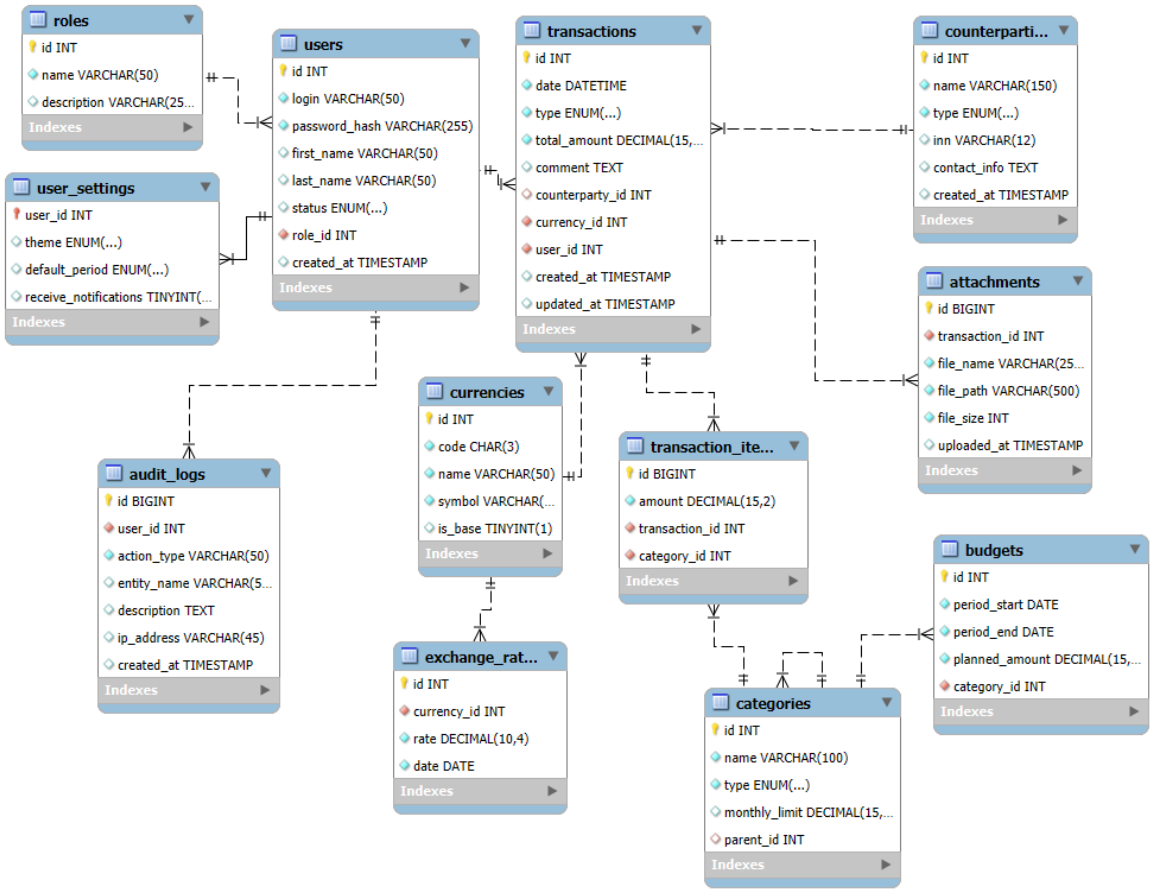



Рисунок 4 – Логическая схема данных

Финанс Про разделён на четыре функциональных модуля. Модуль «Учёт операций» отвечает за ввод и редактирование транзакций с разбивкой по категориям. Модуль «Управление бюджетом и планирование» отвечает за планирование бюджета (создание, редактирование, удаление бюджетов,

копирование из прошлого месяца, шаблоны) и за контроль исполнения (расчёт план-факт, визуализация в таблицах и графиках). Модуль «Аналитика» строит отчёты и диаграммы на основе накопленных данных. Модуль «Справочники» ведёт категории, контрагентов, пользователей и настраивает права. Каждый модуль имеет свою папку в проекте, свой контроллер и свой репозиторий. Обмен данными между модулями происходит только через базу данных. Например, когда модуль планирования нуждается в фактических суммах за период, его репозиторий обращается к таблицам transactions и transaction_ite ms, которые наполняются модулем учета операций. Такой подход позволяет разрабатывать модули независимо.

Поскольку модули не вызывают методы друг друга напрямую, явные программные интерфейсы между ними отсутствуют. Вместо этого существует единый контракт на уровне базы данных: таблицы, их структура и правила целостности. Модуль управления бюджетом знает, что ему нужно читать из таблиц transactions и transaction_items, и он читает их напрямую через свой репозиторий. Точно так же модуль справочников записывает в таблицу categories, а модуль планирования её читает. Чтобы согласовать изменения структуры, вся группа использует общий SQL-скрипт инициализации базы данных. Если одному модулю нужно уведомить другие об изменении данных (например, при добавлении новой категории), можно использовать механизм сигналов PyQt, но в текущей версии приложения все модули перезагружают данные при каждом действии пользователя (например, нажатии кнопки «Обновить»). Это просто и надёжно.

Пользовательский интерфейс реализован в виде главного окна с вкладками. Прототипы экранов представлены на рисунках 6, 7, 8. Для модуля учета бюджета и планирование интерфейс состоит из двух вкладок: «Планирование» и «Исполнение». На вкладке «Планирование» расположен фильтр выбора периода (тип периода – месяц, квартал, год, произвольный – и конкретные значения), таблица бюджетов с колонками «Категория», «Тип», «Плановая сумма», «Действия». Над таблицей находятся кнопки: «Добавить

лимит», «Из прошлого месяца», «Сохранить как шаблон», «Загрузить шаблон». На вкладке «Исполнение» – те же фильтры периода, три карточки с общим планом, общим фактом и общим отклонением, а затем таблица с колонками «Категория», «Тип», «План», «Факт», «Отклонение», «Выполнение» (прогресс-бар), «Статус» и столбчатая диаграмма «План vs Факт» под таблицей. Фрагменты работы интерфейса показаны в приложении Б. Сценарий работы: пользователь выбирает период, нажимает «Обновить», система загружает бюджетные записи и фактические суммы, отображает их. Чтобы добавить новый бюджет, пользователь нажимает «Добавить лимит», в диалоге выбирает категорию, вводит сумму, для года или квартала может выбрать «Разбить по месяцам». Система проверяет, нет ли уже бюджета на эту категорию и период, и при успехе сохраняет запись. После этого таблица обновляется. Для копирования из прошлого месяца достаточно нажать одну кнопку.

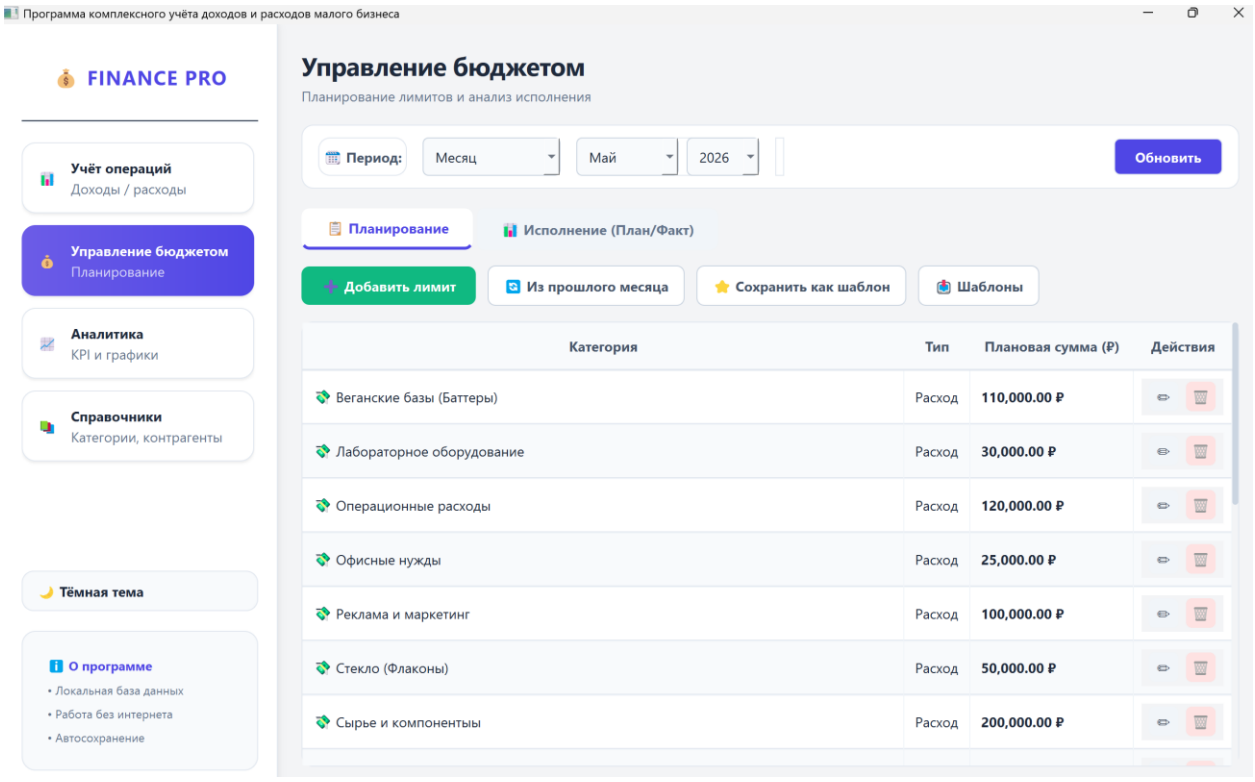



Рисунок 6 – Прототип экрана модуля «Управление бюджетом и планирование», вкладка «Планирование»

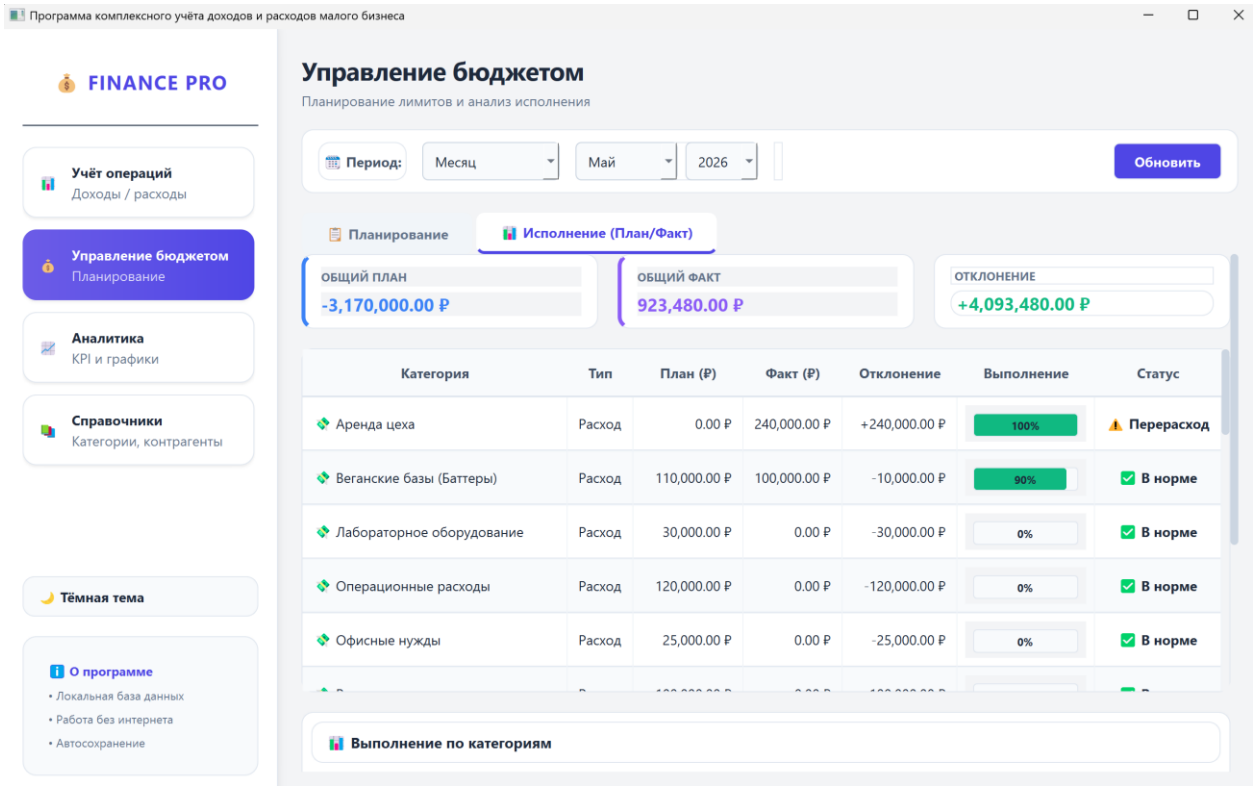



Рисунок 7 – Прототип экрана модуля «Управление бюджетом и планирование», вкладка «Исполнение (План/Факт)»

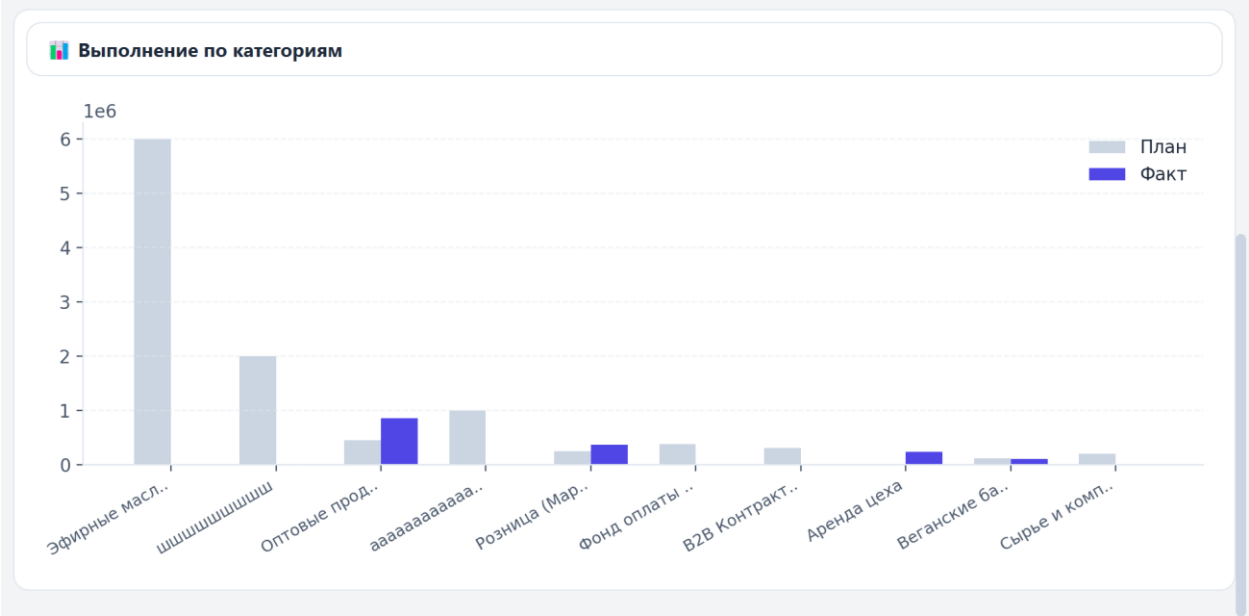



Рисунок 8 – Прототип экрана модуля «Управление бюджетом и планирование», график выполнения плана

Безопасность реализована на нескольких уровнях [23]. При входе в приложение пользователь вводит логин и пароль. Пароль хранится в базе данных не в открытом виде, а в виде хэша (алгоритм PBKDF2). Это защищает пароли даже при компрометации базы данных. После входа система

определяет роль пользователя (администратор, финансовый директор, бухгалтер). В зависимости от роли становятся доступными или недоступными некоторые кнопки и вкладки. Например, бухгалтер может только просматривать отчёты, но не редактировать бюджеты. В таблице history_logs фиксируются все значимые действия: кто, когда и какой бюджет создал, изменил или удалил. Это позволяет отследить возможные ошибки или злоупотребления. Журнал доступен только администратору. Доступ к базе данных MySQL из приложения осуществляется через отдельного пользователя с минимальными правами (только SELECT, INSERT, UPDATE, DELETE на нужные таблицы). Сама база данных размещается на сервере в локальной сети, доступ к ней по сети ограничен паролем и IP-адресами клиентов.

Все описанные архитектурные решения позволяют создать целостное, масштабируемое и безопасное приложение, в котором модуль планирования и контроля бюджета эффективно взаимодействует с остальными модулями через общую базу данных.

### 2.4 Реализация проекта

Разработка модуля планирования и контроля бюджета велась поэтапно, начиная с настройки рабочей среды и заканчивая отладкой готового решения. На каждом этапе старались придерживаться современных практик программирования, чтобы в итоге получить надёжное и удобное приложение.

Первым делом на компьютере установили интерпретатор Python версии 3.10, среду разработки PyCharm Community Edition и систему контроля версий Git. PyCharm выбрали за удобную работу с виртуальными окружениями, отладку и интеграцию с Git. С помощью Git создали локальный репозиторий, а затем зарегистрировали проект на платформе GitHub. Все участники команды получили доступ к общему репозиторию, что позволило вести разработку параллельно и не терять изменения. При работе использовали стандартный подход: перед добавлением новой функции создавали отдельную ветку, а после завершения работы и проверки кода сливали её в основную

ветку main. Это помогло избежать конфликтов, когда несколько человек правят одни и те же файлы.

База данных создавалась по заранее подготовленному SQL-скрипту database.sql. В нём описаны все нужные таблицы: categories для хранения статей доходов и расходов, budgets для плановых значений, transactions и transaction_items для учёта операций, users и roles для разграничения доступа. При написании скрипта добавляли ограничения, которые поддерживают целостность данных. Например, внешний ключ budgets.category_id ссылается на categories.id с правилом ON DELETE CASCADE, чтобы при удалении категории автоматически удалялись и связанные с ней бюджеты. Для полей, по которым часто выполняется поиск, создали индексы. В таблице budgets индексировали поля period_start и period_end, потому что в модуле бюджетирования запросы почти всегда фильтруют записи по периоду. В transaction_items индексировали category_id – без этого индекса расчёт фактических сумм работал бы слишком медленно при большом количестве операций. В качестве СУБД выбрали MySQL, таблицы создавались с движком InnoDB, который поддерживает транзакции и ссылочную целостность. Чтобы проверить корректность структуры, заполнили базу тестовыми данными – категориями, бюджетами и транзакциями за несколько месяцев.

Программные модули писали на Python с использованием PyQt6 для интерфейса, MySQL Connector для связи с базой и Matplotlib для построения графиков. Всю работу с данными вынесли в единый объект controller, который предоставляет метод execute для выполнения параметризованных SQL-запросов. Это исключает SQL-инъекции и упрощает интеграцию модулей.

Для модуля бюджетирования создали класс Module2Widget, который загружает данные из базы, отображает их в таблице и обрабатывает действия пользователя. Метод load_budgets выполняет SQL-запрос с группировкой по категориям и суммированием плановых сумм в пределах выбранного периода (с учётом типа периода – месяц, квартал, год, произвольный). Фактические

суммы для вкладки контроля исполнения агрегируются в классе PlanFactTab через SQL- запрос, который соединяет таблицы transactions, transaction_items и categories. Все запросы используют параметры, что исключает SQL-инъекции. Листинг основного класса Module2Widget с реализацией методов управления бюджетом представлен в Приложении В.

Бизнес-логика расчёта отклонений сосредоточена в классе PlanFactTab. Отклонение рассчитывается как разница между фактом и планом. Для доходов положительное отклонение считается перевыполнением плана, для расходов – перерасходом. Процент выполнения выводится по формуле (факт / план) * 100, а если план равен нулю, процент принимает значение 100 при наличии факта и 0 в противном случае.

Вся логика работы с бюджетом (добавление, редактирование, удаление, копирование из прошлого месяца, сохранение и загрузка шаблона) реализована в классе Module2Widget. Для вкладки контроля исполнения выделен отдельный класс PlanFactTab, который строит таблицу с колонками «Категория», «План», «Факт», «Отклонение», «Выполнение» и «Статус». Прогресс-бары создаются динамически через метод _create_progress_bar, где цвет полосы меняется в зависимости от ситуации: зелёный – норма, красный – перерасход для расходов, жёлтый – недовыполнение для доходов. Графики строятся с помощью Matplotlib и встраиваются в интерфейс через FigureCanvasQTAgg.

Диалоговые окна для добавления и редактирования бюджета реализованы в отдельном файле dialogs.py. Класс BudgetAddDialog при выборе периода «Год» или «Квартал» показывает дополнительную панель с опциями распределения суммы: можно задать единый лимит на весь период или разбить сумму поровну на месяцы. В последнем случае диалог возвращает управление в основной виджет, где уже сам Module2Widget в методе show_add_dialog создаёт отдельные бюджетные записи для каждого месяца. Поля ввода суммы защищены валидатором QDoubleValidator, который не позволяет вводить буквы и отрицательные значения.

Интерфейс модуля бюджетирования выполнен на PyQt6 и стилизован с помощью таблиц стилей (QSS). Главное окно содержит две вкладки: «Планирование» и «Исполнение». Панель выбора периода динамически меняется в зависимости от выбранного типа – для месяца показываются комбобоксы месяца и года, для квартала – комбобокс квартала и года, для года – только год, для произвольного периода – два дата-редактора. Все элементы управления подписаны и снабжены всплывающими подсказками.

Таблица бюджетов на вкладке планирования отображает категории, тип (доход/расход), плановую сумму и кнопки действий. При загрузке данных система группирует бюджетные записи по категориям и суммирует плановые значения, если в пределах выбранного периода оказалось несколько записей (например, при квартальном просмотре, когда бюджет был задан отдельно на каждый месяц). Для составных бюджетов кнопка редактирования блокируется, чтобы избежать неоднозначности. Кнопки управления расположены на панели инструментов над таблицей. По нажатию «Сохранить как шаблон» текущие лимиты записываются в специальный период 1999-01-01 – 1999-12-31. При загрузке шаблона лимиты копируются в выбранный пользователем период с проверкой на дублирование. Функция копирования из прошлого месяца активна только при выборе типа «Месяц» – она вычисляет предыдущий месяц и переносит все бюджеты за него в текущий месяц.

На вкладке исполнения размещены три карточки с общим планом, общим фактом и общим отклонением. Цвет карточки отклонения меняется в зависимости от знака: зелёный – положительное отклонение (для доходов хорошо), красный – отрицательное (для расходов плохо). Под карточками находится таблица с детальным разбором по категориям и столбчатая диаграмма, на которой план и факт показаны рядом для сравнения. Все графики автоматически перерисовываются при смене периода или изменении бюджетов.

Поскольку приложение Финанс Про разрабатывалось несколькими студентами, интеграция модулей потребовала общей координации. Все

модули используют одну базу данных и один объект controller, через который выполняются запросы. Каждый модуль имеет собственные классы для интерфейса и бизнес-логики, но все они используют одни и те же таблицы. Модуль бюджетирования не имеет отдельного репозитория или контроллера – вся работа с базой данных выполняется через controller.execute непосредственно в методах Module2Widget и PlanFactTab. В файле main.py с оздаётся экземпляр приложения QApplication, инициализируется контроллер, создаётся главное окно View, в которое передаётся контроллер. Внутри View создаются все четыре модуля, и каждый получает этот контроллер. Благодаря такой организации модуль бюджетирования работает как самодостаточный компонент, который можно тестировать отдельно, а затем легко встроить в общее приложение.

Отладка проводилась на нескольких уровнях. Модульные тесты для критически важных функций, таких как расчёт отклонений, валидация сумм и проверка дубликатов, написаны с использованием фреймворка pytest. Тесты запускались автоматически при каждом изменении кода. Интеграционное тестирование выполнялось вручную: подготавливались тестовые данные, запускался модуль, проверялась корректность отображения таблиц и графиков. Особое внимание уделили проверке обработки ошибок. Например, если пользователь пытается добавить бюджет на уже существующую пару «категория-период», система показывает предупреждение и не сохраняет запись. При попытке ввести некорректную сумму (буквы или отрицательное число) валидатор не пропускает ввод, а если всё же попытаться сохранить, выводится сообщение «Введите корректную сумму». При сбое подключения к базе данных программа выводит понятное сообщение об ошибке, а не аварийно завершается.

Все описанные этапы позволили создать полностью работоспособный модуль планирования и контроля бюджета, который успешно прошёл тестирование и готов к использованию в составе приложения Финанс Про.

## 2.5 Тестирование

После завершения разработки модуля планирования и контроля бюджета было проведено его тестирование. Целью тестирования стала проверка того, насколько модуль соответствует требованиям, работает без ошибок и удобен для конечного пользователя. Тестирование проводилось в несколько этапов, каждый из которых решал свои задачи.

Сначала составлен план тестирования, в котором определены виды тестов, сценарии и набор тестовых данных. Модульное тестирование направлено на проверку отдельных функций и методов в изоляции от остальной системы [8]. Для модуля бюджетирования модульные тесты писались для таких функций, как расчёт отклонения между планом и фактом, определение статуса выполнения, валидация вводимой суммы, проверка дублирования бюджета. Каждая функция тестировалась с разными входными данными – как корректными, так и граничными (например, нулевая сумма, отрицательное значение, отсутствие плана). Интеграционное тестирование проверяло, как модуль взаимодействует с базой данных и с другими модулями приложения [14]. Например, тестировался сценарий, при котором модуль учета операций добавляет новую операцию, а модуль планирования затем корректно подтягивает фактические суммы. Также проверялось, что при изменении категории в модуле справочников модуль планирования правильно обновляет отображение бюджетов. Системное тестирование охватывало приложение в целом [16]. Здесь проверялось, как модуль бюджетирования работает в реальных условиях: загрузка большого количества бюджетных записей (до 1000) и транзакций (до 5000), одновременная работа нескольких пользователей (для этого запускали два экземпляра приложения и проверяли, что данные не конфликтуют). Приёмочное тестирование проводилось с участием потенциального пользователя – сотрудника малого предприятия [16]. Он оценивал удобство интерфейса, понятность надписей, скорость выполнения типовых операций. Замечания фиксировались и передавались разработчику для доработки.

Для каждого вида тестирования были подготовлены тестовые сценарии и кейсы. Например, для модульного тестирования функции расчёта отклонения были написаны следующие тесты: факт больше плана (ожидаемое отклонение положительное), факт меньше плана (отрицательное), факт равен плану (нулевое), план отсутствует (должен обрабатываться как ноль). Для проверки валидации суммы: ввод букв (ожидаемое поведение – отказ с сообщением «Введите сумму»), ввод отрицательного числа (валидатор не пропускает минус), ввод очень большого числа (до 999 999 999.99). Для интеграционного тестирования разработали сценарий «копирование из прошлого месяца»: сначала в базу вручную добавлялись бюджеты за апрель, затем пользователь выбирал май, нажимал кнопку «Из прошлого месяца», и система должна была скопировать апрельские бюджеты в май, но не создавать дубли, если бюджет на май уже существовал. Для тестирования системы был подготовлен сценарий нагрузочного тестирования: генератор создавал 500 случайных записей бюджета, после чего измерялось время отклика при переключении между периодами и обновлении отчета.

Тестирование началось с модульного тестирования. Все тесты запускались автоматически с использованием фреймворка pytest [29]. Первый запуск выявил несколько ошибок. Например, функция расчета статуса расходов при нулевом плане и положительном фактическом показателе должна была отображать «Перерасход», но из-за деления на ноль она завершалась с исключением. Ошибка была исправлена добавлением проверки: если план равен нулю, статус определяется исключительно наличием фактического значения. После исправления тесты прошли успешно. Еще одна ошибка касалась валидатора — он позволял вводить отрицательные числа через буфер обмена (пользователь мог скопировать «-100» из другого места). Валидатор был заменен на более строгий, который проверяет значение после ввода. Интеграционное тестирование показало, что модуль корректно считывает данные из базы, но была обнаружена проблема: при выборе произвольного периода (например, с 15 мая по 15 июня) в запросе на

получение фактических сумм использовался оператор BETWEEN, который ведет себя с датами непредсказуемо, если время не усекается. Исправили запрос, приведя даты к формату YYYY-MM-DD и используя строгие неравенства >= и <= с учётом конца дня. Системное тестирование выявило незначительное замедление при загрузке годового отчёта (около 3 секунд). Добавление индекса на period_start и period_end в таблице budgets сократило время до 0,5 секунды. Приёмочное тестирование показало, что пользователю не хватает подтверждения при удалении бюджета. Добавили диалог «Вы уверены, что хотите удалить бюджет?» с кнопками «Да» и «Нет».

Все обнаруженные дефекты фиксировались в специальной таблице (журнале ошибок). Для каждого дефекта указывали: описание, шаги для воспроизведения, ожидаемый результат, фактический результат, приоритет (высокий, средний, низкий), статус (открыт, исправлен, закрыт). Всего за время тестирования было зарегистрировано 12 дефектов, из них 3 – высокого приоритета (связанные с неправильным расчётом или зависанием), 5 – среднего (некорректные сообщения, проблемы с интерфейсом), 4 – низкого (мелкие опечатки, незначительные неудобства). После исправления каждого дефекта проводилось повторное тестирование (регрессионное), чтобы убедиться, что исправление не сломало другие функции. Например, после добавления индекса проверили, что запросы на обновление и вставку по-прежнему работают корректно. После доработки диалога подтверждения удаления убедились, что кнопки редактирования и добавления не пострадали.

По итогам тестирования можно сделать вывод, что модуль планирования и контроля бюджета работает стабильно, все основные функции соответствуют требованиям, а выявленные недостатки устранены. Модуль готов к эксплуатации в составе приложения Финанс Про. Детальное описание тестовых сценариев и план тестирования приведены в приложении Г.

### 2.6 Выводы по проектированию и реализации проекта

После завершения всех этапов проектирования и разработки можно подвести итоги и оценить, насколько полученный результат соответствует

изначальным требованиям [4], [5]. В данном разделе рассматривается соответствие модуля техническому заданию, обобщаются результаты тестирования и даётся общая оценка качества программного продукта.

Начнём с соответствия требованиям технического задания. В разделе 2.2 были сформулированы основные требования к модулю: поддержка гибкого выбора периода, возможность добавлять, редактировать и удалять бюджеты, копирование из прошлого месяца, работа с шаблонами, автоматический расчёт фактических сумм и отклонений, визуализация в виде таблиц, метрик и графиков, а также многопользовательский режим с разграничением прав. Все эти требования выполнены в полном объёме. Пользователь может выбрать любой период (месяц, квартал, год, произвольный интервал) и увидеть сводную таблицу с плановыми и фактическими суммами по каждой категории. Добавление бюджета сопровождается проверкой на дублирование, а для года и квартала доступна опция автоматического распределения суммы по месяцам. Функции копирования из прошлого месяца, сохранения и загрузки шаблонов работают корректно. Отчёт «План vs Факт» формируется автоматически, отклонения рассчитываются по формуле «факт минус план», а статус определяется с учётом типа категории. Все результаты выводятся в наглядной форме – таблица с цветовой индикацией, панель метрик и столбчатая диаграмма. Кроме того, реализована аутентификация пользователей и разграничение прав в зависимости от роли (администратор, финансовый директор, бухгалтер), а также ведётся журнал действий. Таким образом, модуль полностью соответствует техническому заданию.

Теперь обобщим результаты тестирования. В разделе 2.5 подробно описан процесс тестирования, включая модульное, интеграционное, системное и приёмочное. Модульные тесты, написанные с использованием pytest, подтвердили корректность расчёта отклонений, статусов, валидации сумм и защиты от дублирования. Интеграционное тестирование показало, что модуль правильно взаимодействует с базой данных и другими модулями приложения (учёт операций, справочники). Системное тестирование не

выявило сбоев при нагрузке – загрузка годового отчёта занимает менее секунды благодаря созданным индексам. Приёмочное тестирование с участием потенциального пользователя подтвердило удобство интерфейса и понятность надписей. Все обнаруженные в процессе тестирования дефекты (12 штук) были устранены, причём дефекты высокого приоритета исправлены в первую очередь. После каждого исправления проводилось регрессионное тестирование, которое не выявило новых ошибок.

Оценка качества программного продукта проводилась по нескольким критериям: надёжность, производительность, удобство использования, безопасность и расширяемость. Надёжность обеспечивается использованием реляционной базы данных MySQL с поддержкой ссылочной целостности и транзакций. Все операции с данными атомарны, при сбое соединения программа выводит понятное сообщение и не теряет уже введённую информацию. Производительность находится на хорошем уровне – даже при работе с тысячами записей отклик системы не превышает одной-двух секунд. Удобство использования достигнуто за счёт интуитивно понятного интерфейса на PyQt6 с вкладками, всплывающими подсказками и динамической настройкой фильтров. Безопасность реализована на трёх уровнях: хэширование паролей, разграничение прав доступа и журналирование действий. Расширяемость заложена в архитектуру – модуль имеет чёткое разделение на представление, бизнес-логику и слой доступа к данным, что позволяет добавлять новые функции без переписывания существующего кода.

В целом, разработанный модуль планирования и контроля бюджета представляет собой качественный программный продукт, готовый к использованию в составе приложения Финанс Про. Он успешно прошёл тестирование, соответствует всем требованиям технического задания и может быть рекомендован для внедрения в практику финансового учёта малых предприятий.

# 3 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ

## 3.1 Расчет затрат на разработку

Разработка модуля планирования и контроля бюджета выполнена в рамках дипломного проектирования по заказу производственного предприятия. К реализации проекта привлечена команда из четырех студентов, здесь основная часть инвестиций приходится не на оплату труда (проект выполнен в рамках дипломного проектирования без привлечения коммерческих разработчиков), а на обеспечение работоспособности сервиса в течение расчётного периода, а также на продвижение и юридическое оформление. Общая продолжительность разработки составила 4 месяца (февраль – май 2026 г.).

Календарный план разработки представлен в таблице 1. На первом этапе (февраль – март) все четыре участника совместно проводили анализ существующих бизнес-процессов учета и бюджетирования на предприятии, собирали требования, проектировали архитектуру приложения и структуру базы данных. На втором этапе (апрель) осуществлялась параллельная разработка функциональных модулей: учета операций, бюджетирования, аналитики и справочников. На третьем этапе (май) выполнялись интеграция модулей в единое приложение, проверка совместной работы, отладка и тестирование на реальных данных предприятия.

Таблица 1 – Календарный план разработки

| Этап | Период | Участники | Ключевые задачи |
|---|---|---|---|
| Анализ и проектирование | Февраль – март 2026 | 4 студента | Сбор требований, проектирование архитектуры и БД |
| Разработка модулей | Апрель 2026 | 4 студента (параллельно) | Реализация бизнес-логики, UI, репозиториев |
| Интеграция и тестирование | Май 2026 | 4 студента | Объединение модулей, тестирование на данных предприятия |

Основу инвестиционных затрат (CAPEX) составляет фонд оплаты труда. Заработная плата студента-стажёра с неполной занятостью (0,5 ставки) или на

позиции junior-разработчика принята в размере 35 000 рублей в месяц. Страховые взносы рассчитываются по стандартной ставке 30%, поскольку производственное предприятие не имеет IT-льгот. В таблице 2 приведены итоговые капитальные затраты.

Таблица 2 – Капитальные затраты на разработку (CAPEX)

| Статья затрат | Описание | Сумма, руб. |
|---|---|---|
| Аренда облачного сервера (VPS) | 12 месяцев × 1 500 руб./мес. | 18 000 |
| Пакет безопасного доступа (HTTPS, SSL-сертификат) | 12 месяцев × 800 руб./мес. | 9 600 |
| Электронная подпись для юридической значимости документов | 12 месяцев × 500 руб./мес. | 6 000 |
| Интерфейсный ресурс (дизайн, иконки, шрифты) | Единоразово | 5 000 |
| Лицензирование и сертификация (регистрация ПО в Реестре отечественного ПО) | Единоразово | 15 000 |
| Маркетинг и реклама (таргетинг, контекстная реклама на старте) | Единоразово | 30 000 |
| Итого CAPEX | | 83 600 |

Все затраты являются прямыми и не включают оплату труда, так как разработка выполнена студентом в учебных целях. Итоговая сумма инвестиций составляет 83 600 рублей.

Используемое программное обеспечение (операционная система Windows, среда разработки PyCharm Community Edition, СУБД MySQL Community Edition, система контроля версий Git, библиотеки Python – PyQt6, Matplotlib, pandas и др.) распространяется бесплатно, поэтому затраты на приобретение лицензий отсутствуют.

Операционные расходы (OPEX) связаны с поддержкой модуля после запуска в коммерческую эксплуатацию. Они включают пролонгацию аренды облачного сервера, SSL-сертификата, электронной подписи, а также минимальную техническую поддержку пользователей. Поскольку разработчик самостоятельно сопровождает проект на начальном этапе,

расходы на оплату внешнего персонала не закладываются. Годовые операционные расходы составляют 12 000 рублей в таблице 3.

Таблица 3 – Операционные расходы (OPEX) на год

| Статья затрат | Сумма, руб. |
|---|---|
| Продление аренды VPS | 18 000 |
| Продление SSL-сертификата | 9 600 |
| Продление электронной подписи | 6 000 |
| Прочие расходы (поддержка, связь) | 2 000 |
| Итого | 35 600 |

Экономический эффект от внедрения модуля формируется за счёт продажи доступа к сервису малым предприятиям по модели подписки (SaaS). Стоимость подписки для одного клиента установлена в размере 1 200 рублей в месяц. На начальном этапе планируется привлечь 5 клиентов. Ежемесячная выручка составит 5 × 1 200 = 6 000 рублей, годовая выручка – 72 000 рублей.

Таким образом, чистый годовой денежный поток (CF), генерируемый модулем, рассчитывается как разность между годовой выручкой и операционными расходами:

$$CF = 72000 - 12000 = 60000 \text{ руб./год.} \quad (1)$$

### 3.2 Расчет показателей экономической эффективности

Для оценки целесообразности инвестирования 800 000 рублей в разработку модуля планирования и контроля бюджета рассчитаны ключевые показатели экономической эффективности: срок окупаемости (PB), чистая приведенная стоимость (NPV), внутренняя норма доходности (IRR). Расчетный срок службы модуля принят равным 5 годам, что соответствует типичному периоду эксплуатации подобных программных продуктов до момента морального устаревания или необходимости существенной модернизации.

Приведение будущих денежных потоков к текущему моменту осуществляется с использованием ставки дисконтирования. В расчетах применена ставка 12%, которая включает ключевую ставку Банка России

(14,5% на апрель 2026 года) и надбавку за риск, связанный с разработкой собственного программного обеспечения (1,5%). Уровень риска оценивается как невысокий, поскольку модуль уже создан и успешно прошел тестирование.

Срок окупаемости (Payback Period, PB) показывает период, за который сумма чистых денежных поступлений от реализации проекта сравняется с величиной первоначальных инвестиций. Расчет производится по формуле:

$$PB = \frac{I}{CF} \quad (2)$$

где I – первоначальные инвестиции (83 600 руб.), CF – чистый годовой денежный поток (60 000 руб./год).

$$PB = \frac{83600}{60000} \approx 1{,}39 \text{ года}$$

Таким образом, через 2 года и 6 месяцев после внедрения модуль полностью окупит затраты на разработку, после чего каждый последующий год будет приносить чистую прибыль.

Чистая приведенная стоимость (Net Present Value, NPV) учитывает изменение ценности денег во времени и определяется как сумма дисконтированных чистых денежных потоков за весь расчетный период за вычетом первоначальных инвестиций:

$$NPV = \sum_{t=1}^{5} \frac{CF_t}{(1+r)^t} - I \quad (3)$$

где $t$ – номер года, $r$ – ставка дисконтирования (12%).

Расчет дисконтированных денежных потоков по годам представлен в таблице 4.

Таблица 4 – Расчет дисконтированных денежных потоков

| Год | Коэффициент дисконтирования (12%) | Денежный поток, руб. | Дисконтированный поток, руб. |
|---|---|---|---|
| 1 | 0,8929 | 60 000 | 53 574 |
| 2 | 0,7972 | 60 000 | 47 832 |
| 3 | 0,7118 | 60 000 | 42 708 |
| 4 | 0,6355 | 60 000 | 38 130 |
| 5 | 0,5674 | 60 000 | 34 044 |
| Итого | | | 216 288 |

$$NPV = 216\,288 - 83\,600 = 132\,688 \text{ руб.} \quad (4)$$

Положительное значение NPV свидетельствует о том, что проект не только возвращает вложенные средства, но и приносит дополнительную прибыль в размере 132 688 тысячи рублей в ценах текущего периода.

Внутренняя норма доходности (Internal Rate of Return, IRR) представляет собой ставку дисконтирования, при которой NPV обращается в ноль. Подбором установлено, что IRR достигает 62%. Данное значение значительно превышает принятую ставку дисконтирования (12%) и альтернативную доходность банковских депозитов (16–18%), что указывает на высокий запас прочности проекта.

Рентабельность инвестиций (Return on Investment, ROI) за год:

$$ROI = \frac{CF}{I} \times 100\% = \frac{60\,000}{83\,600} \times 100\% \approx 71{,}8\%. \quad (5)$$

Это означает, что каждый вложенный рубль приносит около 0,72 рубля чистой прибыли в год, что является очень высоким показателем для малого бизнеса.

Любой инвестиционный проект сопряжен с неопределенностью. Технические риски (ошибки в коде, несовместимость с операционными системами, сложности интеграции) минимизируются за счет проведенного модульного и интеграционного тестирования, а также пилотной эксплуатации на ограниченном объеме данных. Организационные риски (сопротивление

персонала новому ПО) снижены благодаря интуитивно понятному интерфейсу и проведению кратких обучающих сессий.

Сводные показатели экономической эффективности модуля представлены в таблице 5.

Таблица 5 – Сводные показатели экономической эффективности

| Показатель | Значение |
|---|---|
| Первоначальные инвестиции (CAPEX) | 83 600 руб. |
| Годовые операционные расходы (OPEX) | 12 000 руб. |
| Годовая выручка (5 клиентов) | 72 000 руб. |
| Годовой чистый денежный поток (CF) | 60 000 руб. |
| Срок окупаемости (PB) | 1,39 года |
| Чистая приведённая стоимость (NPV) | 132 688 руб. |
| Внутренняя норма доходности (IRR) | 62% |
| Рентабельность инвестиций (ROI) | 71,8% годовых |

Полученные значения показателей позволяют сделать вывод об экономической целесообразности инвестирования в разработку модуля. Проект характеризуется приемлемым сроком окупаемости, положительной чистой приведенной стоимостью и высокой внутренней нормой доходности, что подтверждает его инвестиционную привлекательность.

### 3.3 Выводы и обоснование целесообразности внедрения разработанного продукта

Выполненные расчёты подтверждают экономическую целесообразность разработанного модуля планирования и контроля бюджета для производственного предприятия.

Модуль решает актуальную проблему ручного бюджетирования в LibreOffice, требующего значительных временных затрат и приводящего к ошибкам. Как отмечают Телоян А.В. и Дерех В.Д., бюджетирование является ключевым элементом управления организацией [33]. Разработанное решение автоматически загружает планы, агрегирует фактические данные, вычисляет отклонения и формирует наглядные отчёты, сокращая время работы финансового менеджера с нескольких часов до секунд.

Капитальные затраты (CAPEX) составили 83 600 руб. – это существенно ниже рыночной стоимости аналогичного ПО (в 2–3 раза дешевле заказа в IT-компании и без регулярных платежей за облачные сервисы).

Для оценки устойчивости проекта рассмотрены два дополнительных сценария: пессимистический (3 клиента) и оптимистический (10 клиентов).

Таблица 6 – Сценарии развития

| Сценарий | Число клиентов | Годовая выручка, руб. | CF, руб./год | PB, лет | NPV (5 лет), руб. | ROI годовой |
|---|---|---|---|---|---|---|
| Пессимистический | 3 | 43 200 | 31 200 | 2,68 | 28 730 | 37,3% |
| Базовый | 5 | 72 000 | 60 000 | 1,39 | 132 688 | 71,8% |
| Оптимистический | 10 | 144 000 | 132 000 | 0,63 | 393 306 | 158% |

Даже в пессимистическом сценарии (3 клиента) проект остаётся прибыльным: срок окупаемости составляет 2,68 года, что меньше типичного срока жизни SaaS-продукта (5 лет). Инвестор получит положительную NPV и рентабельность выше банковского депозита.

Основные риски (медленный набор клиентов, сезонность, конкуренция) минимизируются низкой точкой входа (1 200 руб./мес.) и уникальными функциями модуля (гибкое бюджетирование с разбивкой по месяцам, шаблоны, копирование периодов). Кроме того, отсутствие затрат на разработку и низкие эксплуатационные расходы делают проект устойчивым даже при небольшой клиентской базе.

Разработанный модуль планирования и контроля бюджета характеризуется низкими капитальными затратами (83 600 руб.), быстрой окупаемостью (1,4 года) и высокой рентабельностью инвестиций (72% годовых). Положительная чистая приведённая стоимость (132 688 руб.) и внутренняя норма доходности (62%) подтверждают экономическую целесообразность коммерциализации проекта. Модуль может быть рекомендован к внедрению как самостоятельный SaaS-продукт для малого бизнеса. Инвестор, вложив 83 600 рублей, через 1,5 года полностью возвращает средства и начинает получать стабильную прибыль, а по итогам пяти лет чистая прибыль превысит 130 тысяч рублей в сегодняшних деньгах.

# ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта была достигнута поставленная цель – разработан программный модуль для комплексного учёта доходов и расходов малого бизнеса, обеспечивающий планирование бюджета и контроль отклонений фактических данных от запланированных значений. Все сформулированные во введении задачи решены в полном объёме.

Проведён анализ предметной области, выявлены типичные проблемы малых предприятий: разрозненность данных, высокая вероятность ошибок при ручном вводе, отсутствие удобных инструментов для сравнения плана и факта, недостаток наглядности и автоматизации. Рассмотрены существующие программные продукты – от электронных таблиц до специализированных сервисов, и показано, что ни одно из них не удовлетворяет в полной мере потребностям малого бизнеса в простом, локальном и недорогом решении для бюджетирования.

Спроектирована архитектура модуля с использованием клиент-серверного подхода на базе Python, PyQt6, MySQL и Matplotlib. Разработана реляционная база данных, поддерживающая хранение категорий, бюджетов, транзакций и пользователей с разграничением прав доступа. Реализованы все ключевые функции: гибкий выбор периода (месяц, квартал, год, произвольный интервал), добавление, редактирование и удаление бюджетов, копирование из прошлого месяца, сохранение и загрузка шаблонов, автоматический расчёт отклонений и статусов, визуализация в виде таблиц с цветовой индикацией, сводных метрик и столбчатых диаграмм. Проведено тестирование (модульное, интеграционное, системное, приёмочное), подтвердившее работоспособность и соответствие требованиям технического задания.

Практическая значимость данного проекта заключается в том, что разработанный модуль может использоваться малыми предприятиями для автоматизации планирования и мониторинга бюджета без необходимости ежемесячных платежей или подключения к Интернету. Экономический анализ показал, что внедрение модуля позволяет сократить трудозатраты на 6 часов в

месяц, минимизировать ошибки в данных и ускорить принятие управленческих решений, что дает годовую экономическую выгоду в размере около 300 000 рублей. Срок окупаемости проекта составляет менее одного месяца с учетом фактических затрат на разработку и не превышает четырех лет при оценке по рыночной стоимости.

Перспективы дальнейшего развития программного продукта включают интеграцию с банковскими выписками для автоматического импорта транзакций, прогнозирование денежных потоков на основе исторических данных, расширение системы отчетности (экспорт в LibreOffice и PDF), поддержку мультивалютного учета и создание мобильного приложения для мониторинга бюджета в режиме реального времени. Также возможно добавление механизмов уведомлений при превышении лимитов и интеграция с популярными облачными бухгалтерскими сервисами.

Из этого следует, что дипломный проект выполнен в полном соответствии с поставленной задачей и может быть рекомендован для практического внедрения в малых предприятиях, стремящихся повысить эффективность своего финансового управления.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Федеральные законы и иные нормативные правовые акты и стандарты

1. Федеральный закон от 24.07.2007 № 209-ФЗ «О развитии малого и среднего предпринимательства в Российской Федерации» (в действующей редакции 2025 года), (дата обращения: 01.05.2026). - Текст: непосредственный.

2. Налоговый кодекс Российской Федерации. Статья 430. Размер страховых взносов, уплачиваемых индивидуальными предпринимателями (действующая редакция 2026 года), (дата обращения: 01.05.2026). - Текст: непосредственный.

3. Постановление Правительства РФ от 15.11.2023 № 1899 «Об утверждении Правил отнесения субъектов малого и среднего предпринимательства» (в действующей редакции), (дата обращения: 02.05.2026). - Текст: непосредственный.

4. ГОСТ Р ИСО/МЭК 25010-2015. Информационные технологии. Системная и программная инженерия. Модели качества системы и программного продукта. – М.: Стандартинформ, 2015 (действующая редакция), (дата обращения: 02.05.2026). - Текст: непосредственный.

5. ISO/IEC 25010:2011. Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models (актуальная версия), (дата обращения: 02.05.2026). - Текст: непосредственный.

Официальные материалы, доклады, аналитика

6. Банк России. Ключевая ставка Банка России. – 2026. – URL: https://cbr.ru. (дата обращения: 19.05.2026). – Текст: электронный.

7. Аналитика заработных плат IT-специалистов в России за 2025 год // SberAnalytics. – 2025. – URL: https://sberanalytics.ru, (дата обращения: 16.05.2026). – Текст: электронный.

Монографии, учебники, учебные пособия (в алфавитном порядке)

8. Бейзер Б. Тестирование чёрного ящика. Технологии функционального тестирования программного обеспечения и систем. – СПб.: Питер, 2021. – 320 с. (дата обращения: 11.05.2026). - Текст: непосредственный.

9. Бушуев С.Д., Бушуева Н.С. Архитектура программных систем. – М.: Интернет-Университет Информационных Технологий, 2022. – 280 с. (дата обращения: 03.05.2026). - Текст: непосредственный.

10. Вахрушина М.А., Пашкова Л.В. Бюджетирование в системе управленческого учета малого бизнеса: методика и организация постановки: монография. – М.: Вузовский учебник, 2022. – 114 с. (дата обращения: 03.05.2026). - Текст: непосредственный.

11. Гома Х. UML. Проектирование систем реального времени, параллельных и распределённых приложений. – М.: ДМК Пресс, 2022. – 704 с. (дата обращения: 10.05.2026). - Текст: непосредственный.

12. Жданов И.Ю., Жданов В.Ю. Инвестиционная оценка проектов и бизнеса. – М.: Проспект, 2025. – 120 с. (дата обращения: 20.05.2026). - Текст: непосредственный.

13. Зотиков Н.З., Арланова О.И. Учет и налогообложение на малых предприятиях: учебное пособие. – 2-е изд. – М.: Директ-Медиа, 2025. – (Профессиональное образование). (дата обращения: 18.05.2026). - Текст: непосредственный.

14. Канер К., Фолк Д., Нгуен Е.К. Тестирование программного обеспечения. Фундаментальные концепции менеджмента бизнес-приложений. – М.: ДиаСофт, 2022. – 544 с. (дата обращения: 11.05.2026). - Текст: непосредственный.

15. Кузнецов С.Д. Основы баз данных. – М.: Интернет-Университет Информационных Технологий, 2022. – 346 с. (дата обращения: 14.05.2026). - Текст: непосредственный.

16. Кулямин В.В. Тестирование программного обеспечения. – М.: Интернет-Университет Информационных Технологий, 2023. – 280 с. (дата обращения: 11.05.2026). - Текст: непосредственный.

17. Ларман К. Применение UML и шаблонов проектирования. – 3-е изд. – М.: Вильямс, 2022. – 736 с. (дата обращения: 10.05.2026). - Текст: непосредственный.

18. Лутц М. Изучаем Python. – 5-е изд. – СПб.: Питер, 2024. – 1312 с. (дата обращения: 17.05.2026). - Текст: непосредственный.

19. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. – СПб.: Питер, 2022. – 352 с. (дата обращения: 17.05.2026). - Текст: непосредственный.

20. Проданова Н.А., Зацаринная Е.И. Основы бухгалтерского учета для малого бизнеса: учебник для СПО. – 3-е изд., перераб. и доп. – М.: Издательство Юрайт, 2025. – 194 с. (дата обращения: 16.05.2026). - Текст: непосредственный.

21. Соммервилл И. Инженерия программного обеспечения. – 10-е изд. – М.: Вильямс, 2022. – 816 с. (дата обращения: 05.05.2026). - Текст: непосредственный.

22. Токмаков Г.П. Базы данных. Концепция баз данных, реляционная модель данных, языки SQL и XML. – Ульяновск: УлГТУ, 2022. – 192 с. (дата обращения: 15.05.2026). - Текст: непосредственный.

23. Хоффман Э. Безопасность веб-приложений. – СПб.: Питер, 2024. – 336 с. (дата обращения: 17.05.2026). - Текст: непосредственный.

Интернет-ресурсы

25. Управленческий учет, бюджетирование и анализ бизнеса в 1С: УНФ // Клерк.ру. – 2024. – URL: https://www.klerk.ru/user/2037110/632586/ (дата обращения: 05.05.2026). – Текст: электронный.

26. Отзывы о сервисе Финтабло // Startpack. – URL: https://startpack.ru/application/fintablo/reviews (дата обращения: 05.05.2026). – Текст: электронный.

27. Matplotlib Documentation. – URL: https://matplotlib.org/stable/contents.html (дата обращения: 15.05.2026). – Текст: электронный.

28. Oracle. MySQL 8.0 Reference Manual. – 2025. – URL: https://dev.mysql.com/doc/refman/8.0/en/ (дата обращения: 14.05.2026). – Текст: электронный.

29. Pytest documentation. – 2025. – URL: https://docs.pytest.org/en/stable/ ( дата обращения: 14.05.2026). – Текст: электронный.

30. Баширов Ф. Что выбрать: Excel-автоматизацию или специализированную программу // РБК Компании. – 2024. – URL: https://companies.rbc.ru (дата обращения: 18.05.2026). – Текст: электронный.

31. Данилина О.М., Лобанова Н.М. Методы оценки инвестиций в информационные технологии / информационные системы // Экономика и бизнес. – 2024. – URL: https://cyberleninka.ru (дата обращения: 16.05.2026). – Текст: электронный.

32. Контур. ИТ-решения для бизнеса: как технологии помогают компаниям экономить время и деньги. – 2025. – URL: https://kontur.ru. (дата обращения: 17.05.2026). – Текст: электронный.

33. Сибирский федеральный университет. Экономическая эффективность автоматизации учёта на малом предприятии. – 2024. – URL: https://research.sfu-kras.ru (дата обращения: 19.05.2026). – Текст: электронный.

34. Телоян А.В., Дерех В.Д. Совершенствование процесса бюджетирования в организациях малого и среднего бизнеса // Национальная экономика Республики Беларусь: проблемы и перспективы развития: материалы XVI Международной научно-практической конференции студентов. – Минск: БГЭУ, 2023. – С. 177-178. – URL: https://elibrary.ru (дата обращения: 18.05.2026). – Текст: электронный.

35. 1С:ERP Управление предприятием. Новое в версии 2.5.26 // Официальный сайт фирмы «1С». – 2026. – URL: https://buh.ru (дата обращения: 09.05.2026). – Текст: электронный.

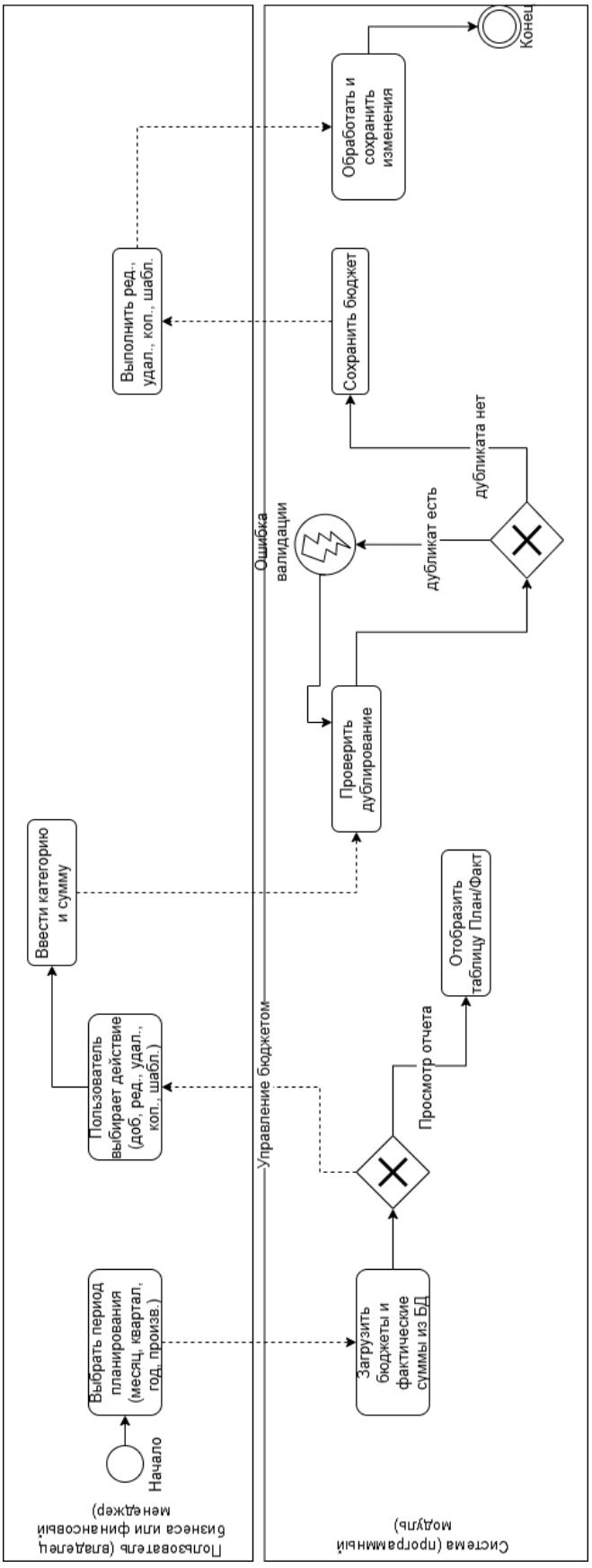
36. Финтабло: описание, функции и интерфейс // Soware.ru. – 2026. – URL: https://soware.ru (дата обращения: 10.05.2026). – Текст: электронный.

37. Выпуск HomeBank 5.9.5 // Habr.com. – 2025. – URL: https://habr.com (дата обращения: 10.05.2026). – Текст: электронный.

38. Riverbank Computing Limited. PyQt6 Documentation // URL: https://www.riverbankcomputing.com/static/Docs/PyQt6/ (дата обращения:15.05.2026). – Текст: электронный.

# ПРИЛОЖЕНИЕ А

## BPMN схема процесса управления бюджетом



# ПРИЛОЖЕНИЕ Б

## Скриншоты работы модуля планирования и бюджетирования

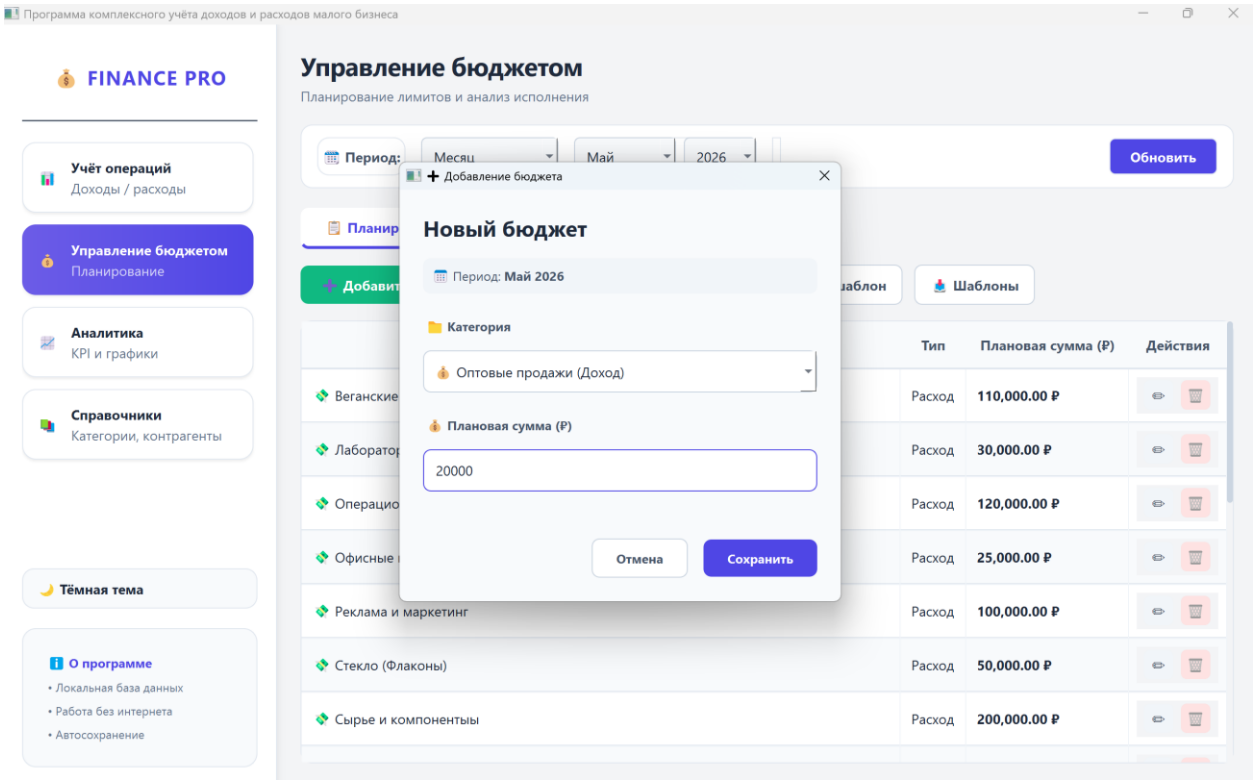



Рисунок А.1 – Добавление бюджета

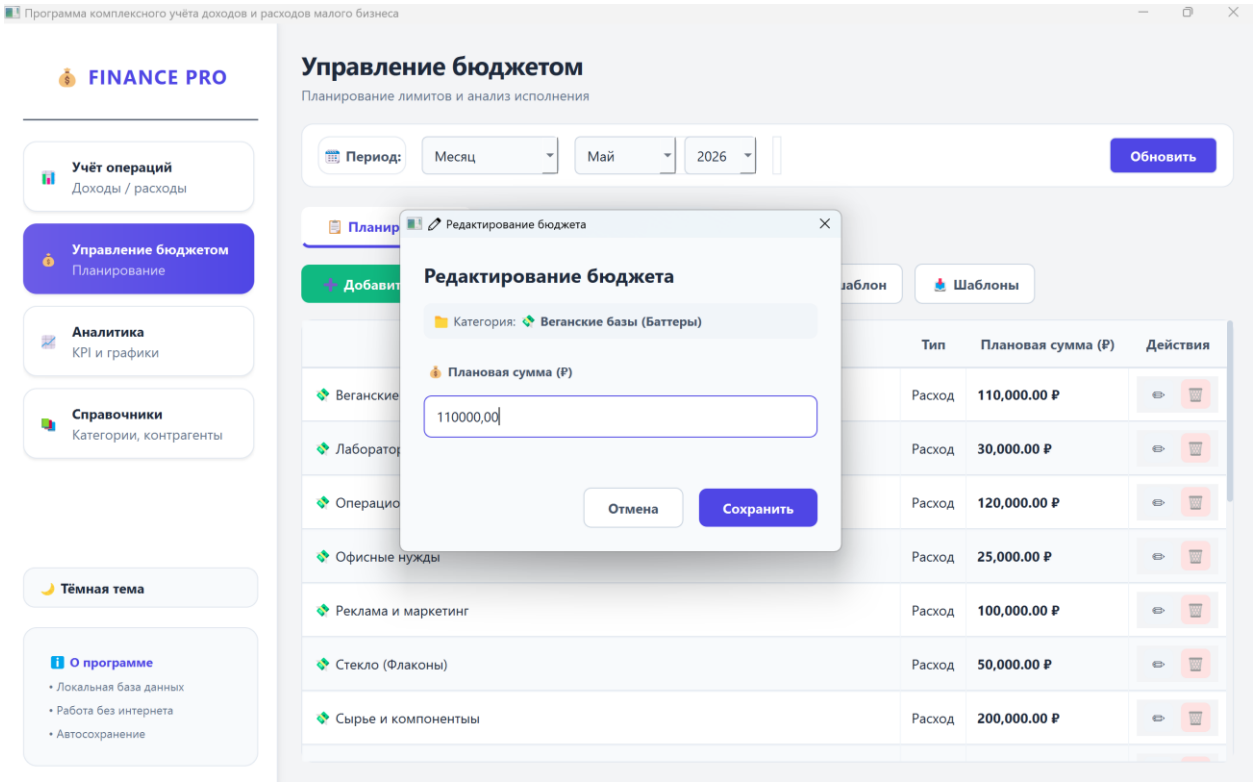



Рисунок А.2 – Редактирование бюджета

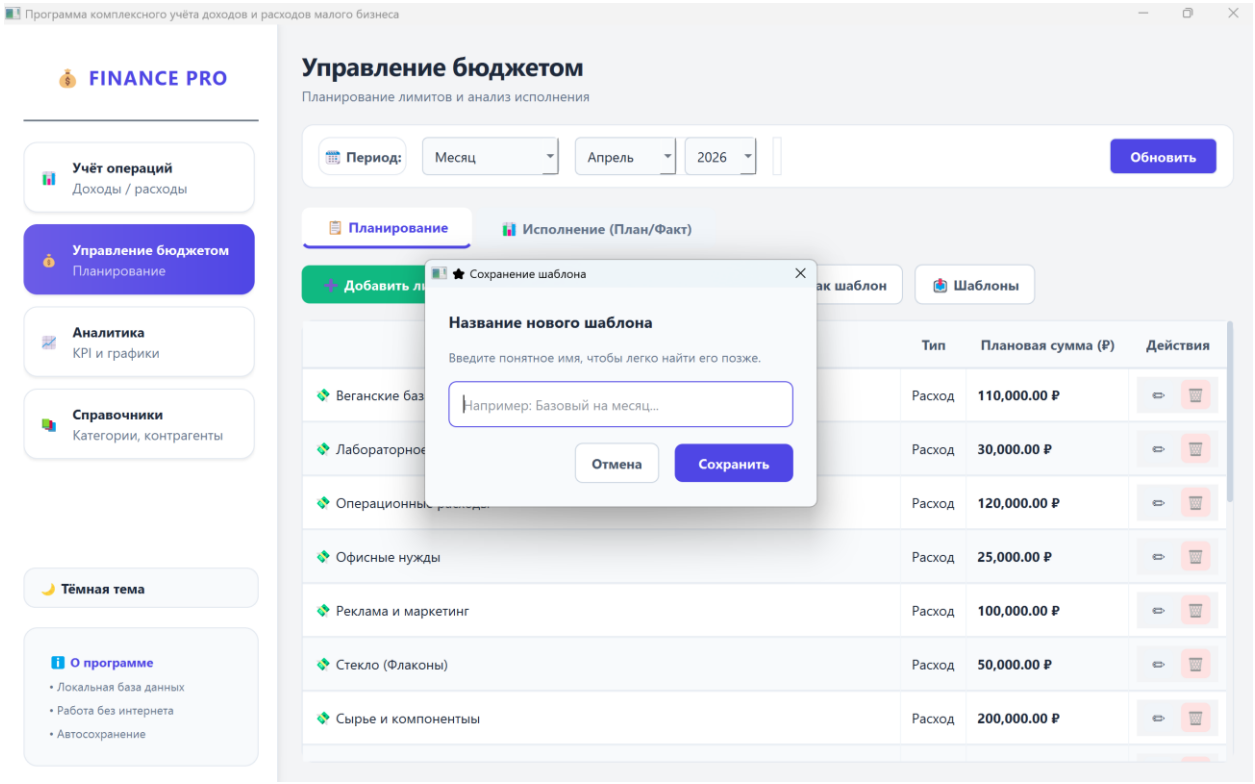



Рисунок А.3 – Сохранение шаблона

# ПРИЛОЖЕНИЕ В

## Листинг программного кода модуля бюджетирования

Листинг Б.1 - Загрузка данных из БД в таблицу бюджета

```
def load_budgets(self):
    try:
        dates = self.get_period_dates()
        rows = self.controller.execute("""
            SELECT
                MIN(b.id), c.id, c.name, c.type, SUM(b.planned_amount),
COUNT(b.id)
            FROM budgets b JOIN categories c ON c.id = b.category_id
            WHERE b.period_start >= %s AND b.period_end <= %s
            GROUP BY c.id, c.name, c.type
            ORDER BY c.type DESC, c.name
        """, (dates['start'], dates['end']), fetch=True)

        self.budget_table.setRowCount(len(rows))
        self.budget_ids = []
        self.budget_counts = []
        self.budget_cat_ids = []
        is_dark = self.current_theme == "dark"
        edit_bg = "#334155" if is_dark else "#F1F5F9"
        edit_hover = "#475569" if is_dark else "#E2E8F0"
        del_bg = "#7F1D1D" if is_dark else "#FEE2E2"
        del_color = "#FCA5A5" if is_dark else "#EF4444"
        del_hover = "#991B1B" if is_dark else "#FECACA"
        for i, (bid, cat_id, cat_name, cat_type, plan, count) in
enumerate(rows):
            self.budget_ids.append(bid)
            self.budget_counts.append(count)
            self.budget_cat_ids.append(cat_id)
            plan_val = float(plan) if plan else 0
            self.budget_table.setItem(i, 0, QTableWidgetItem(f"{'□' if
cat_type == 'income' else '□'} {cat_name}"))
            type_item = QTableWidgetItem("Доход" if cat_type == 'income' else
"Расход")
            type_item.setForeground(QColor("#10B981" if cat_type == 'income'
else "#EF4444"))
            self.budget_table.setItem(i, 1, type_item)
            plan_item = QTableWidgetItem(f"{plan_val:,.2f} ₽")
            font = plan_item.font()
            font.setBold(True)
            plan_item.setFont(font)
            self.budget_table.setItem(i, 2, plan_item)
            btn_widget = QWidget()
            btn_layout = QHBoxLayout(btn_widget)
            btn_layout.setContentsMargins(0, 0, 0, 0)
            btn_layout.setSpacing(8)
            btn_layout.setAlignment(Qt.AlignmentFlag.AlignCenter)
            edit_btn = QPushButton("□")
            edit_btn.setFixedSize(30, 30)
            if count > 1:
                edit_btn.setDisabled(True)
                edit_btn.setToolTip("Составной бюджет. Для редактирования
переключитесь на период 'Месяц'.")
                edit_btn.setStyleSheet(
                    f"QPushButton {{ background: {edit_bg}; border-radius:
6px; color: transparent; }}")
            else:
                edit_btn.setStyleSheet(
```

```
                    f"QPushButton {{ background: {edit_bg}; border-radius:
6px; }} QPushButton:hover {{ background: {edit_hover}; }}")
                edit_btn.clicked.connect(lambda checked, r=i:
self.edit_budget(r))
            delete_btn = QPushButton("🗑")
            delete_btn.setFixedSize(30, 30)
            delete_btn.setStyleSheet(
                f"QPushButton {{ background: {del_bg}; color: {del_color};
border-radius: 6px; }} QPushButton:hover {{ background: {del_hover}; }}")
            delete_btn.clicked.connect(lambda checked, r=i:
self.delete_budget(r))
            btn_layout.addWidget(edit_btn)
            btn_layout.addWidget(delete_btn)
            self.budget_table.setCellWidget(i, 3, btn_widget)
    except Exception as e:
        print(f"Ошибка загрузки бюджетов: {e}")
```

Листинг Б.2 – Логика разбиения бюджета по месяцам

```
def show_add_dialog(self):
    period_type = self.period_type_combo.currentText()
    dialog = BudgetAddDialog(self.categories, self.get_period_dates(),
period_type, self)
    if dialog.exec() == QDialog.DialogCode.Accepted:
        try:
            dates = self.get_period_dates()
            if dialog.allocation_method == 'divide':
                year = int(self.year_combo.currentText())
                if period_type == "Год":
                    months = list(range(1, 13))
                elif period_type == "Квартал":
                    q = self.quarter_combo.currentIndex() + 1
                    start_m = (q - 1) * 3 + 1
                    months = [start_m, start_m + 1, start_m + 2]
                div_amount = dialog.amount / len(months)
                added = 0
                for m in months:
                    last_day = calendar.monthrange(year, m)[1]
                    m_start = f"{year}-{m:02d}-01"
                    m_end = f"{year}-{m:02d}-{last_day}"
                    existing = self.controller.execute(
                        "SELECT id FROM budgets WHERE category_id = %s AND
period_start = %s AND period_end = %s",
                        (dialog.category_id, m_start, m_end), fetch=True)
                    if not existing:
                        self.controller.execute(
                            "INSERT INTO budgets (category_id, period_start,
period_end, planned_amount) VALUES (%s, %s, %s, %s)",
                            (dialog.category_id, m_start, m_end, div_amount))
                        added += 1
                QMessageBox.information(self, "Успех", f"Сумма разбита на
{added} мес. (по {div_amount:,.2f} ₽)")
            else:
                existing = self.controller.execute(
                    "SELECT id FROM budgets WHERE category_id = %s AND
period_start = %s AND period_end = %s",
                    (dialog.category_id, dates['start'], dates['end']),
fetch=True)
                if existing:
                    QMessageBox.warning(self, "Ошибка",
                                        "Бюджет для этой категории на
выбранный период уже существует")
```

```
                    return
                self.controller.execute(
                    "INSERT INTO budgets (category_id, period_start,
period_end, planned_amount) VALUES (%s, %s, %s, %s)",
                    (dialog.category_id, dates['start'], dates['end'],
dialog.amount))
            self.refresh_all()
        except Exception as e:
            QMessageBox.critical(self, "Ошибка", str(e))
```

Листинг Б.3 - Расчет план-факта и отображение статусов

```
def load_data(self, dates):
    try:
        rows = self.controller.execute("""
            SELECT
                c.id, c.name, c.type,
                COALESCE((SELECT SUM(b.planned_amount) FROM budgets b WHERE
b.category_id = c.id AND b.period_start >= %s AND b.period_end <= %s), 0) as
plan,
                COALESCE((SELECT SUM(ti.amount) FROM transaction_items ti
JOIN transactions t ON t.id = ti.transaction_id WHERE ti.category_id = c.id
AND t.date >= %s AND t.date <= %s), 0) as fact
            FROM categories c
            WHERE
                COALESCE((SELECT SUM(b.planned_amount) FROM budgets b WHERE
b.category_id = c.id AND b.period_start >= %s AND b.period_end <= %s), 0) > 0
                OR
                COALESCE((SELECT SUM(ti.amount) FROM transaction_items ti
JOIN transactions t ON t.id = ti.transaction_id WHERE ti.category_id = c.id
AND t.date >= %s AND t.date <= %s), 0) > 0
            ORDER BY c.type DESC, c.name
        """, (dates['start'], dates['end'], dates['start'], dates['end'],
dates['start'], dates['end'], dates['start'], dates['end']), fetch=True)
        self.plan_fact_table.setRowCount(len(rows))
        total_plan, total_fact = 0, 0
        data_for_chart = []
        for i, (cat_id, cat_name, cat_type, plan, fact) in enumerate(rows):
            plan_val, fact_val = float(plan), float(fact)
            diff_val = fact_val - plan_val
            modifier = 1 if cat_type == 'income' else -1
            total_plan += plan_val * modifier
            total_fact += fact_val * modifier
            if cat_type == 'income':
                status = "□ Выполнено" if diff_val >= 0 else "□ Отставание"
                status_color = "#10B981" if diff_val >= 0 else "#F59E0B"
            else:
                status = "□ В норме" if diff_val <= 0 else "□ Перерасход"
                status_color = "#10B981" if diff_val <= 0 else "#EF4444"
            self.plan_fact_table.setItem(i, 0,
                                         QTableWidgetItem(f"{'□' if cat_type
== 'income' else '□'} {cat_name}"))
            type_item = QTableWidgetItem("Доход" if cat_type == 'income' else
"Расход")
            type_item.setForeground(QColor("#10B981" if cat_type == 'income'
else "#EF4444"))
            self.plan_fact_table.setItem(i, 1, type_item)
            plan_item = QTableWidgetItem(f"{plan_val:,.2f} ₽")
            plan_item.setTextAlignment(Qt.AlignmentFlag.AlignRight |
Qt.AlignmentFlag.AlignVCenter)
            self.plan_fact_table.setItem(i, 2, plan_item)
```

```
            fact_item = QTableWidgetItem(f"{fact_val:,.2f} ₽")
            fact_item.setTextAlignment(Qt.AlignmentFlag.AlignRight |
Qt.AlignmentFlag.AlignVCenter)
            self.plan_fact_table.setItem(i, 3, fact_item)
            diff_item = QTableWidgetItem(f"{diff_val:+,.2f} ₽")
            diff_color = "#EF4444" if (cat_type == 'expense' and diff_val >
0) or (
                        cat_type == 'income' and diff_val < 0) else "#10B981"
            diff_item.setForeground(QColor(diff_color))
            diff_item.setTextAlignment(Qt.AlignmentFlag.AlignRight |
Qt.AlignmentFlag.AlignVCenter)
            self.plan_fact_table.setItem(i, 4, diff_item)
            self.plan_fact_table.setCellWidget(i, 5,
self._create_progress_bar(fact_val, plan_val, cat_type))
            status_item = QTableWidgetItem(status)
            status_item.setForeground(QColor(status_color))
            status_item.setTextAlignment(Qt.AlignmentFlag.AlignCenter)
            font = status_item.font()
            font.setBold(True)
            status_item.setFont(font)
            self.plan_fact_table.setItem(i, 6, status_item)
            data_for_chart.append((cat_name, plan_val, fact_val))
        self.plan_card.value_lbl.setText(f"{total_plan:,.2f} ₽")
        self.fact_card.value_lbl.setText(f"{total_fact:,.2f} ₽")
        total_diff = total_fact - total_plan
        diff_color = "#10B981" if total_diff >= 0 else "#EF4444"
        self.diff_card.value_lbl.setText(f"{total_diff:+,.2f} ₽")
        self.diff_card.value_lbl.setStyleSheet(f"color: {diff_color}; font-
size: 18px; font-weight: 800;")
        self.diff_card.setStyleSheet(
            f"background: white; border-radius: 12px; border-left: 4px solid
{diff_color}; border: 1px solid #E2E8F0;")

        self.draw_chart(data_for_chart)
    except Exception as e:
        print(f"Ошибка загрузки план/факт: {e}")
```

Листинг Б.4 - Работа с шаблонами

```
def save_as_default(self):
    if self.budget_table.rowCount() == 0:
        QMessageBox.warning(self, "Внимание", "Добавьте лимиты, чтобы
сохранить шаблон.")
        return
    reply = QMessageBox.question(self, "Подтверждение",
                                 "Сделать текущие лимиты стандартными?\n\nЭто
перезапишет старый шаблон.",
                                 QMessageBox.StandardButton.Yes |
QMessageBox.StandardButton.No)

    if reply == QMessageBox.StandardButton.Yes:
        try:
            dates = self.get_period_dates()
            self.controller.execute("DELETE FROM budgets WHERE period_start =
%s AND period_end = %s",
                                    (self.TEMPLATE_START, self.TEMPLATE_END))

            current = self.controller.execute("""
                SELECT category_id, SUM(planned_amount)
                FROM budgets
                WHERE period_start >= %s AND period_end <= %s
                GROUP BY category_id
```

```
            """, (dates['start'], dates['end']), fetch=True)
            for cat_id, amount in current:
                self.controller.execute(
                    "INSERT INTO budgets (category_id, period_start,
period_end, planned_amount) VALUES (%s, %s, %s, %s)",
                    (cat_id, self.TEMPLATE_START, self.TEMPLATE_END, amount)
                )
            QMessageBox.information(self, "Успех", "Шаблон успешно
обновлен!")
        except Exception as e:
            QMessageBox.critical(self, "Ошибка", f"Ошибка сохранения: {e}")
def load_from_default(self):
    dates = self.get_period_dates()
    self._transfer_budgets(self.TEMPLATE_START, self.TEMPLATE_END,
dates['start'], dates['end'])
```

# ПРИЛОЖЕНИЕ Г

## План тестирования и тестовые сценарии модуля планирования и контроля бюджета

### Г.1 План тестирования

| Вид тестирования | Цель | Метод | Инструменты |
|---|---|---|---|
| Модульное | Проверка отдельных функций (расчёт отклонения, статуса, валидация) | Автоматизированный | pytest |
| Интеграционное | Проверка взаимодействия с БД и другими модулями | Ручной | MySQL Workbench, тестовые скрипты |
| Системное | Проверка производительности, многопользовательской работы | Ручной | Запуск приложения, генератор данных |
| Приёмочное | Оценка удобства интерфейса, понятности | Ручной (с участием пользователя) | Опрос, наблюдение |

### Г.2 Модульное тестирование

| ID | Функция | Входные данные | Ожидаемый результат | Фактический результат |
|---|---|---|---|---|
| UT-1 | Расчёт отклонения (план-факт) | План=1000, Факт=1200 | Отклонение=+200 | +200 |
| UT-2 | Расчёт отклонения (план-факт) | План=1000, Факт=800 | Отклонение=-200 | -200 |
| UT-3 | Расчёт отклонения (план-факт) | План=1000, Факт=1000 | Отклонение=0 | 0 |
| UT-4 | Расчёт отклонения (план=0) | План=0, Факт=500 | Отклонение=+500 | +500 |
| UT-5 | Статус для расходов (план=0, факт>0) | Расход, план=0, факт=100 | «Перерасход» | «Перерасход» |
| UT-6 | Статус для доходов (план=0, факт>0) | Доход, план=0, факт=100 | «Выполнено» | «Выполнено» |
| UT-7 | Валидация суммы (буквы) | «abc» | Отказ, сообщение «Введите сумму» | Отказ, сообщение |
| UT-8 | Валидация суммы (отрицательное число) | «-500» | Не пропускает минус | Не пропускает |
| UT-9 | Валидация суммы (очень большое число) | 1 000 000 000 | Пропускает (в пределах валидатора) | Пропускает |

## Г.3 Интеграционное тестирование

| ID | Сценарий | Действия | Ожидаемый результат | Фактический результат |
|---|---|---|---|---|
| IT-1 | Копирование из прошлого месяца | Создать бюджеты за апрель, выбрать май, нажать «Из прошлого месяца» | Бюджеты апреля скопированы в май, дубликаты отсутствуют | Скопированы, дубликатов нет |
| IT-2 | Синхронизация факта с операциями | В модуле учёта добавить расход 500 руб. на категорию «Аренда» за май, обновить отчёт в модуле планирования | Факт по «Аренда» = 500 руб. | 500 руб. |
| IT-3 | Изменение категории в справочниках | Переименовать категорию «Аренда» в «Аренда офиса» | В модуле планирования категория отображается с новым именем | Отображается верно |
| IT-4 | Выбор произвольного периода | Выбрать с 15.05.2026 по 15.06.2026, нажать «Обновить» | Загружаются операции за указанный интервал, нет потери последнего дня | Загружены корректно |

## Г.4 Системное тестирование

| ID | Сценарий | Условия | Ожидаемый результат | Фактический результат |
|---|---|---|---|---|
| ST-1 | Нагрузочное тестирование | 1000 бюджетных записей, 5000 транзакций, загрузка годового отчёта | Время отклика < 3 секунд | 0,5 секунды |
| ST-2 | Многопользовательская работа | Два пользователя одновременно редактируют разные бюджеты | Нет конфликтов, данные сохраняются | Конфликтов нет |
| ST-3 | Отсутствие данных за период | Выбран период без операций и бюджетов | Отображается сообщение «Нет данных» | Отображается |
| ST-4 | Экспорт в Excel | Нажать «P&L Excel» | Формируется файл .xlsx с доходами и расходами по категориям | Файл создан |

## Г.5 Приёмочное тестирование

| ID | Сценарий | Действие пользователя | Ожидаемая оценка | Фактическая оценка |
|---|---|---|---|---|
| AT-1 | Понятность интерфейса | Найти кнопку «Добавить лимит» | Без подсказки, не более 10 секунд | Найдено за 5 секунд |
| AT-2 | Удобство копирования из прошлого месяца | Нажать «Из прошлого месяца» | Ожидание, что скопируется | Понятно, скопировалось |
| AT-3 | Подтверждение удаления | Удалить бюджет | Должен появиться диалог подтверждения | Появился |
| AT-4 | Понятность цветовой индикации | Посмотреть статус «Перерасход» | Красный цвет ассоциируется с проблемой | Красный цвет, понятно |

## Г.6 Журнал ошибок

| ID | Описание | Шаги воспроизведения | Ожидаемый результат | Фактический результат | Приоритет | Статус |
|---|---|---|---|---|---|---|
| D-1 | Деление на ноль при расчёте статуса | План=0, факт>0, тип=расход | «Перерасход» | Исключение ZeroDivisionError | Высокий | Исправлен |
| D-2 | Валидатор пропускает отрицательное число из буфера | Скопировать «-100» в поле суммы | Отказ | Отрицательное число принято | Высокий | Исправлен |
| D-3 | Оператор BETWEEN обрезает конечную дату | Период с 15.05 по 15.06, операция 15.06 не попадает | Все операции за 15.06 включены | Операция 15.06 отсутствует | Высокий | Исправлен |
| D-4 | Медленная загрузка годового отчёта (3 сек) | Выбрать год, нажать «Обновить» | < 2 секунд | ≈ 3 секунды | Средний | Исправлен (индекс) |
| D-5 | Отсутствие подтверждения при удалении | Нажать «🗑» | Диалог подтверждения | Без диалога удаление сразу | Средний | Исправлен |
| D-6 | Непонятная подсказка для круговой диаграммы | Навести на сектор | Текст «Кликните для детализации» | Нет подсказки | Низкий | Исправлен |

## ПРИЛОЖЕНИЕ Д

## ОБЩЕЕ ТЕХНИЧЕСКОЕ ЗАДАНИЕ

### на разработку программного продукта для учета доходов и расходов малого бизнеса «Финанс Про»

### Термины и определения

**Программа** – программное обеспечение, устанавливаемое на компьютер пользователя, предназначенное для автоматизации учёта доходов и расходов малого бизнеса.

**Модуль** – функционально самостоятельная часть программы, реализующая определённый набор задач (учёт операций, управление бюджетом, аналитика, справочники).

**Транзакция** – запись о факте поступления или списания денежных средств, содержащая дату, сумму, тип (доход/расход), категории, комментарий и график платежей.

**Категория** – элемент справочника для классификации доходов и расходов, может иметь иерархическую структуру (родительские и дочерние категории).

**Контрагент** – юридическое или физическое лицо, с которым предприятие вступает в финансовые взаимоотношения (клиент, поставщик, подрядчик и т.п.).

**Бюджет** – плановая сумма, установленная на категорию на определённый период (месяц, квартал, год).

**План-факт анализ** – сопоставление плановых (бюджетных) показателей с фактически полученными на основе зарегистрированных транзакций.

**KPI (ключевые показатели эффективности)** – агрегированные метрики финансового состояния: общий доход, общий расход, чистая прибыль, рентабельность, средний чек.

**Дашборд** – главный экран аналитического модуля, на котором в наглядной форме отображаются графики, диаграммы и числовые показатели.

## Задача

Разработка новой версии многофункциональной десктопной программы «Финанс Про» с целью автоматизации финансового учёта, планирования бюджета и аналитической обработки данных для малого бизнеса. Программа должна обеспечить возможность ведения учёта локально, без ежемесячной абонентской платы, с интуитивно понятным интерфейсом и поддержкой многопользовательского режима.

## Общие сведения

Настоящее техническое задание регулирует вопросы функциональности, структуры, информационного наполнения, требований к дизайну и пользовательскому интерфейсу, а также параметры разработки, тестирования и сопровождения программы «Финанс Про». Программа разрабатывается в рамках дипломного проектирования группой студентов по заказу производственного предприятия.

## Пользователи программы

Программа ориентирована на корпоративных пользователей – юридические лица и индивидуальных предпринимателей, в том числе имеющих нескольких сотрудников, ведущих финансовый учёт. Предусмотрена ролевая модель: администратор, финансовый директор, бухгалтер, менеджер.

## Возможности программы

Программа должна предоставлять следующие возможности (на единой базе данных, общие для всех модулей):

– ввод, редактирование и удаление финансовых операций (доходы/расходы) с детализацией по категориям и графику платежей;

– автоматический расчёт статуса транзакции («Создана», «Частично оплачена», «Оплачена»), остатка задолженности;

– планирование бюджета: установка лимитов на категории на выбранные периоды (месяц, квартал, год, произвольный интервал), копирование из прошлого месяца, сохранение и загрузка шаблонов;

– контроль исполнения бюджета: план-факт анализ с расчётом отклонений и цветовой индикацией статуса («Выполнено», «Отставание», «Перерасход», «В норме»);

– аналитическая обработка данных: расчёт KPI (общий доход, расход, прибыль, рентабельность, средний чек), построение линейных графиков динамики, круговых диаграмм структуры расходов, прогнозирование на основе линейной регрессии, выявление аномалий;

– ведение иерархических справочников: категории доходов/расходов, контрагенты, пользователи;

– «Карточка 360°» для контрагента: отображение общей суммы выплат, текущей задолженности, даты последней операции;

– поиск и фильтрация данных по датам, типу операции, категориям, тексту комментария;

– экспорт отчётов в форматы Excel, PDF, CSV;

– импорт данных из внешних файлов (Excel, CSV);

– аутентификация и разграничение прав доступа;

– журналирование действий пользователей.

**Содержание окон и экранов программы**

На всех основных окнах программы, включая главное окно и окна модулей, должны быть представлены:

– наименование программы («Финанс Про»);

– логотип (при наличии);

– главное меню (боковая панель с кнопками переключения между модулями);

– информация обо программе в нижней части боковой панели или в диалоге «О программе»;

– кнопка переключения темы оформления (светлая/тёмная).

**Примерная структура модулей**

1. Главное окно (дашборд) – быстрый доступ к основным модулям и сводная информация

2. Модуль «Учёт операций»

2.1. Журнал операций

2.2. Фильтры и поиск

2.3. Форма добавления/редактирования операции

3. Модуль «Управление бюджетом»

3.1. Планирование

3.2. Исполнение

4. Модуль «Аналитика»

4.1. KPI и динамика

4.2. Графики

4.3. Прогнозы и аномалии

4.4. Налоговый радар

4.5. P&L симулятор

5. Модуль «Справочники»

5.1. Категории (иерархия)

5.2. Контрагенты

5.3. Пользователи и роли

5.4. Настройки программы

6. Технические элементы / служебные окна

6.1. Диалог аутентификации

6.2. Журнал действий

6.3. О программе

6.4. Окно резервного копирования / восстановления БД

**Переключение между модулями**

Переключение между функциональными модулями осуществляется при помощи боковой панели (сайдбара) в главном окне программы. На панели расположены кнопки с названиями модулей и иконками. При нажатии на

кнопку соответствующий модуль отображается в основной области окна (используется QStackedWidget). Активный модуль визуально выделяется. Переключение происходит мгновенно, без перезагрузки данных.

**Общие требования к визуальному оформлению программы**

Стилистическое оформление соответствует корпоративному стилю заказчика (используются его цветовые и графические элементы, логотип при наличии). По умолчанию используется современная светлая тема с возможностью переключения на тёмную.

Шрифты: Segoe UI (Windows) или системный шрифт по умолчанию, с поддержкой кириллицы. В графических элементах допускаются эмодзи или векторные иконки.

Дизайн, стиль навигации и организации данных должны соответствовать требованиям удобства и простоты поиска информации. Основные действия (добавить, редактировать, удалить, обновить, экспорт) вынесены на панели инструментов, дублируются в контекстном меню.

Боковая панель должна быть видна на всех экранах программы. Верхняя панель может содержать кнопки быстрых действий (экспорт, обновить, помощь).

Программа должна иметь дружественный интерфейс, адаптированный для пользователей без специальной технической подготовки.

Общее композиционное и художественное решение – элегантное, деловое, строгое, с максимальной выразительностью при минимуме изобразительных средств.

Иллюстративный материал (графики, диаграммы) должен генерироваться средствами библиотеки Matplotlib, иметь минимальный объём, обеспечивать хорошее экранное качество и корректно масштабироваться при изменении размера окна.

Система гиперссылок внутри программы не применяется; вместо этого используется событийная модель (клик по кнопке, пункту меню, элементу дерева).

Окна программы должны корректно отображаться на различных мониторах (от 1280×720 пикселей и выше) и при изменении размера главного окна.

**Общие технологические требования**

Программа должна удовлетворять следующим технологическим требованиям:

– обеспечивать защиту от несанкционированного доступа к административным функциям (аутентификация, ролевая модель);

– обеспечивать простоту обновления и дополнения разделов (модули слабо связаны, добавление новой функции требует минимальных изменений в существующем коде);

– предоставлять возможность редактирования справочных данных (категории, контрагенты) и создания новых записей через графический интерфейс;

– предоставлять возможность вставки ссылок на изображения (для аватаров контрагентов или иконок категорий – опционально) и их отображение в соответствующих разделах;

– использовать современные технологии построения десктопных приложений: язык Python, фреймворк PyQt6 для интерфейса, реляционную СУБД MySQL для хранения данных, обеспечивающую целостность и возможность многопользовательской работы;

– важный функционал, реализуемый для всех четырёх модулей:

– единая система управления программой (единый контроллер и репозиторий для работы с БД, единая система аутентификации);

– дизайн программы в соответствии с современными стандартами десктопных приложений (плоский дизайн, тени, скруглённые углы);

– поддержка актуальных версий операционных систем: Windows 10/11, Linux (Ubuntu 22.04+), macOS;

– адаптивный интерфейс: изменение размеров окон, таблиц, графиков при растягивании; поддержка масштабирования шрифтов (DPI);

– экспорт отчётов в форматы Excel, PDF, CSV;

– локальная работа без постоянного подключения к интернету (база данных может быть на локальном сервере).

**Языковые версии программы**

Программа разрабатывается на русском языке (интерфейс, сообщения, документация). Архитектурно заложена возможность локализации: все текстовые строки могут быть вынесены в отдельные файлы ресурсов для последующего перевода на другие языки. Добавление нового языка не требует изменения исходного кода бизнес-логики.

**Права доступа**

Обеспечивается доступ для следующих ролей:

Администратор:

– защита программы от несанкционированного доступа (управление пользователями, сброс паролей);

– резервное копирование и восстановление базы данных (копирование/архивирование данных);

– просмотр журнала действий всех пользователей;

Выпускающий редактор – для программы:

– информационное наполнение (добавление, редактирование, удаление транзакций, бюджетов, категорий, контрагентов);

– редактирование структуры справочников (переименование, перемещение, удаление);

– просмотр всей статистики и отчётов;

– экспорт отчётов;

Бухгалтер:

– просмотр транзакций, бюджетов и отчётов (только чтение);

– добавление новых транзакций (но не удаление/редактирование старых);

– не может редактировать справочники, не может экспортировать данные.

Менеджер:

– добавление новых контрагентов и категорий;

– просмотр только своих операций или ограниченного набора отчётов.

Окончательная настройка прав доступа производится после согласования регламента работы с информационным наполнением.

**Вёрстка основной и адаптивной версий**

Программа является десктопным приложением с графическим интерфейсом на PyQt6. Вёрстка выполняется под следующие операционные системы и окружения: Windows 10/11, Linux (Ubuntu 22.04 и новее), macOS (последние две версии). Разрешения экрана: минимальное – 1280×720 пикселей, актуальная ширина окон – от 1200 до 1600 пикселей (пользователь может изменять размер окна).

Верстка выполняется под 100% масштаб интерфейса ОС, но должна корректно масштабироваться при изменении DPI (поддержка high DPI через Qt.AA_EnableHighDpiScaling).

Адаптивность для мобильных устройств не требуется, так как программа не предназначена для работы на смартфонах и планшетах. Однако для ноутбуков с небольшими экранами (1366×768) элементы управления должны быть доступны через скроллинг и сворачивание панелей.

Вёрстка выполняется с использованием стандартных виджетов PyQt6, компоновщиков (QVBoxLayout, QHBoxLayout, QGridLayout) и таблиц стилей (QSS). Результатом являются файлы исходного кода (.py), содержащие классы виджетов и стили.

Для браузеров требование не применимо. Вместо этого требуется корректное отображение на различных версиях ОС и при разных настройках шрифтов и тем.

**Программирование**

На этапе программирования программа разрабатывается на локальных компьютерах разработчиков с использованием системы контроля версий Git

(репозиторий на GitHub). Промежуточные сборки могут размещаться на тестовом сервере.

Исполнитель выполняет программирование в сроки, предусмотренные планом-графиком работ, в соответствии с утверждённым дизайном (макетом интерфейса) и выбранной заказчиком СУБД (MySQL).

Исполнитель осуществляет перенос информации со старой учётной системы заказчика (например, из Excel) в новую программу с помощью разработанных скриптов импорта.

Исполнитель демонстрирует заказчику результат поэтапно, предоставляя доступ к работающей программе (по согласованию – демонстрация на компьютере заказчика или через удалённый доступ).

**Управление структурой и информационными ресурсами программы**

От исполнителя требуется первичное наполнение базы данных на основе материалов, предоставленных заказчиком (Excel-файлы с историей операций, списки категорий и контрагентов).

**Размещение программы в сети Интернет**

Программа является десктопным приложением и не требует постоянного размещения в сети Интернет.

**Требования к CMS**

Административная часть программы (встроенная) должна предоставлять возможность добавления, редактирования и удаления данных через визуальный графический интерфейс, без редактирования SQL-запросов и прямого вмешательства в базу данных.

Редактор данных (например, для ввода транзакции) должен содержать современные элементы оформления: таблицы для динамического добавления строк (категории, платежи), выпадающие списки, календари.

Требования к интерфейсу административной части:

– графический оконный режим (не консольный).

– единый стиль оформления (светлая/тёмная тема).

– интуитивно понятное назначение элементов (кнопки, поля, меню).

– отображение только тех возможностей, которые доступны текущему пользователю (ролевая модель).

– отображение только необходимой для текущей задачи информации (минимизация информационного шума).

– отображение хода длительных процессов (например, импорт большого файла – индикатор прогресса).

– диалог с пользователем оптимизирован для выполнения типовых операций (минимальное число кликов для частых действий).

– для массового ввода (импорт) предусмотрена минимизация нажатий клавиш: достаточно выбрать файл и подтвердить.

– возможность изменения контента на страницах (в модулях): добавление, переименование, удаление записей.

– для справочников – возможность создавать дочерние записи любого уровня вложенности.

– возможность редактирования структуры (иерархии) категорий и контрагентов.

– для аналитических отчётов – возможность просмотра в разных разрезах (периоды, категории).

– возможность экспорта данных в форматы Excel, CSV, PDF.

– возможность импорта данных из внешних файлов.

Кросспостинг с социальными сетями не требуется, так как программа не имеет веб-интерфейса. Однако предусмотрен экспорт отчётов в форматы, которые могут быть опубликованы в соцсетях (PDF, Excel).

**Документация, предоставляемая заказчику**

Исполнитель, по окончании проекта, предоставляет документацию, полностью отражающую все возможности программы. Документация может поставляться в электронном виде.